

Python & Data Structures and Algorithms — Complete Interview Preparation Guide

A mini-textbook for cracking coding interviews at top tech companies.

Covers Python internals, every major data structure, 14+ algorithm patterns, 150+ curated problems, a 12-week study plan, and quick-reference cheat sheets.

Table of Contents

- [Part 1: Python Mastery for Interviews](#)
 - [Part 2: Data Structures — Deep Dive](#)
 - [Part 3: Algorithm Patterns — The Core of Interview Prep](#)
 - [Part 4: Curated Problem List \(150+ Problems\)](#)
 - [Part 5: Study Plan](#)
 - [Part 6: Quick Reference](#)
-

Part 1: Python Mastery for Interviews

1.1 Python Internals

How Lists Work Internally (Dynamic Arrays)

Python's `list` is implemented as a **dynamic array** — a contiguous block of memory holding pointers to Python objects.

Key internals:

- Internally stored as a C array of `PyObject*` pointers.
- When the array is full and a new element is appended, Python **over-allocates** by a growth factor (roughly 1.125x for large lists). This gives **amortized $O(1)$** append.
- The over-allocation pattern (from CPython source): `new_allocated = (newsize >> 3) + (newsize < 9 ? 3 : 6) + newsize`
- Insertions at the beginning or middle are **$O(n)$** because all subsequent elements must be shifted.

Memory layout of a Python list:

```
list object
+-----+
| ob_refcnt |
| ob_type   |
| ob_size (length) | --> 4
| allocated | --> 8 (capacity, may be > length)
| ob_item  -----> [ ptr0 | ptr1 | ptr2 | ptr3 | NULL | NULL | NULL | NULL ]
+-----+
                        |   |   |   |
                        v   v   v   v
                        obj_0 obj_1 obj_2 obj_3
```

Time complexities:

Operation	Average Case	Worst Case	Notes
<code>append(x)</code>	$O(1)^*$	$O(n)$	*Amortized; $O(n)$ when reallocation
<code>pop()</code>	$O(1)$	$O(1)$	Remove from end
<code>pop(0)</code>	$O(n)$	$O(n)$	Shifts all elements
<code>insert(i, x)</code>	$O(n)$	$O(n)$	Shifts elements after index i
<code>x in list</code>	$O(n)$	$O(n)$	Linear scan
<code>list[i]</code>	$O(1)$	$O(1)$	Direct pointer arithmetic
<code>list[i:j]</code>	$O(j-i)$	$O(j-i)$	Creates new list
<code>sort()</code>	$O(n \log n)$	$O(n \log n)$	Timsort (hybrid merge+insertion sort)

How Dicts Work Internally (Hash Tables)

Python dicts use an **open-addressing hash table** with a compact layout (since Python 3.6+).

Key internals:

- Two arrays: a **sparse hash table** (indices) and a **dense entries array** (key-value pairs in insertion order).
- Hash collisions are resolved using **open addressing with perturbation** (a probe sequence that uses higher bits of the hash).

- Load factor threshold: $\sim 2/3$. When exceeded, the table is resized (typically doubled).
- Since Python 3.7, dicts are **guaranteed to maintain insertion order**.

Dict internal structure (Python 3.7+):

Sparse index table (hash $\rightarrow$ position):

```
[ - | 1 | - | 0 | - | 2 | - | - ]
```

Dense entries array (insertion order):

index	hash	key	value
0	hash("a")	"a"	1
1	hash("b")	"b"	2
2	hash("c")	"c"	3

Time complexities:

Operation	Average	Worst	Notes
<code>d[key]</code>	$O(1)$	$O(n)$	Worst case: all keys collide
<code>d[key] = val</code>	$O(1)$	$O(n)$	Amortized due to resizing
<code>key in d</code>	$O(1)$	$O(n)$	Hash lookup
<code>del d[key]</code>	$O(1)$	$O(n)$	Marks entry as dummy
<code>len(d)</code>	$O(1)$	$O(1)$	Stored as attribute

String Immutability and Implications

Python strings are **immutable sequences** of Unicode code points.

Implications:

- Every "modification" creates a **new string object**.
- String concatenation in a loop (`s += char`) is **$O(n^2)$** in the worst case. Use `''.join(parts)` instead — $O(n)$.
- CPython **interns** short strings and single characters, so `"abc" is "abc"` may be `True` (but never rely on this).
- Slicing creates new string objects: `s[1:5]` is $O(k)$ where k = length of slice.

```
# BAD:  $O(n^2)$  string building
s = ""
for char in data:
    s += char # creates new string each time

# GOOD:  $O(n)$  string building
parts = []
for char in data:
    parts.append(char)
s = ''.join(parts)

# BEST:  $O(n)$  with generator
s = ''.join(char for char in data)
```

1.2 Python Modules Interviewers Expect You to Know

`collections` Module

`Counter`

A dict subclass for counting hashable objects.

```

from collections import Counter

# Basic usage
words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
count = Counter(words)
# Counter({'apple': 3, 'banana': 2, 'cherry': 1})

count.most_common(2)      # [('apple', 3), ('banana', 2)]
count['apple']             # 3
count['missing']          # 0 (no KeyError!)

# Arithmetic
c1 = Counter(a=3, b=1)
c2 = Counter(a=1, b=2)
c1 + c2                   # Counter({'a': 4, 'b': 3})
c1 - c2                   # Counter({'a': 2}) (drops zero/negative)
c1 & c2                   # Counter({'a': 1, 'b': 1}) (min)
c1 | c2                   # Counter({'a': 3, 'b': 2}) (max)

# Common interview use: check if two strings are anagrams
def is_anagram(s, t):
    return Counter(s) == Counter(t)

```

defaultdict

Dict subclass with a factory for missing keys.

```

from collections import defaultdict

# Group items
graph = defaultdict(list)
edges = [(1, 2), (1, 3), (2, 4)]
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)
# {1: [2, 3], 2: [1, 4], 3: [1], 4: [2]}

# Count items (like Counter but more flexible)
freq = defaultdict(int)
for char in "hello":
    freq[char] += 1

# Nested defaultdicts
tree = lambda: defaultdict(tree)
taxonomy = tree()
taxonomy['Animal']['Mammal']['Dog'] = True

```

deque (Double-Ended Queue)

Optimized for $O(1)$ appends and pops from both ends.

```

from collections import deque

dq = deque([1, 2, 3])
dq.appendleft(0)      # O(1) – vs O(n) for list.insert(0, x)
dq.append(4)          # O(1)
dq.popleft()          # O(1) – vs O(n) for list.pop(0)
dq.pop()              # O(1)
dq.rotate(1)          # [3, 1, 2] – rotate right
dq.rotate(-1)         # [1, 2, 3] – rotate left

# BFS template using deque
def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Fixed-size sliding window
window = deque(maxlen=3)
for x in [1, 2, 3, 4, 5]:
    window.append(x)
# window = deque([3, 4, 5], maxlen=3)

```

OrderedDict

Dict that remembers insertion order (explicit guarantees + extra methods).

```

from collections import OrderedDict

# LRU Cache implementation
class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key) # mark as recently used
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False) # remove oldest

```

namedtuple

Lightweight, immutable data class.

```

from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
p.x, p.y      # 3, 4
x, y = p      # unpacking works
p._asdict()   # {'x': 3, 'y': 4}

```

heapq Module (Min-Heap)

Python only provides a **min-heap**. For max-heap, negate values.

```
import heapq

# Basic operations
heap = []
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
heapq.heappush(heap, 4)
heapq.heappush(heap, 1)
heapq.heappop(heap)      # 1 (smallest)
heap[0]                  # 1 (peek without pop)

# Heapify existing list – O(n)
nums = [5, 3, 8, 1, 2]
heapq.heapify(nums)      # [1, 2, 8, 5, 3] (heap property)

# N largest / smallest – O(n log k)
heapq.nlargest(3, nums)   # [8, 5, 3]
heapq.nsmallest(2, nums)  # [1, 2]

# Max-heap trick: negate values
max_heap = []
for val in [3, 1, 4]:
    heapq.heappush(max_heap, -val)
largest = -heapq.heappop(max_heap) # 4

# Custom comparators using tuples
# Tuples are compared element by element
tasks = [(2, "low"), (1, "high"), (3, "med")]
heapq.heapify(tasks)
heapq.heappop(tasks)      # (1, 'high') – sorted by first element

# For objects: wrap in tuple (priority, tie_breaker, object)
import itertools
counter = itertools.count()
heapq.heappush(heap, (priority, next(counter), task_object))
```

bisect Module (Binary Search on Sorted Lists)

```
import bisect

sorted_list = [1, 3, 5, 7, 9]

# bisect_left: leftmost position to insert (before existing equal elements)
bisect.bisect_left(sorted_list, 5)    # 2 (index of 5)
bisect.bisect_left(sorted_list, 4)    # 2 (would go before 5)

# bisect_right: rightmost position to insert (after existing equal elements)
bisect.bisect_right(sorted_list, 5)   # 3 (after existing 5)

# insort: insert while maintaining sort order – O(n) due to shifting
bisect.insort(sorted_list, 4)          # [1, 3, 4, 5, 7, 9]

# Finding if element exists in sorted list – O(log n)
def binary_search(arr, target):
    i = bisect.bisect_left(arr, target)
    return i < len(arr) and arr[i] == target

# Finding count of element in sorted list
def count_occurrences(arr, target):
    left = bisect.bisect_left(arr, target)
    right = bisect.bisect_right(arr, target)
    return right - left
```

itertools Module

```

import itertools

# permutations – all orderings
list(itertools.permutations([1, 2, 3]))
# [(1,2,3), (1,3,2), (2,1,3), (2,3,1), (3,1,2), (3,2,1)]

list(itertools.permutations([1, 2, 3], 2)) # length-2 permutations
# [(1,2), (1,3), (2,1), (2,3), (3,1), (3,2)]

# combinations – unordered selections
list(itertools.combinations([1, 2, 3, 4], 2))
# [(1,2), (1,3), (1,4), (2,3), (2,4), (3,4)]

# combinations_with_replacement
list(itertools.combinations_with_replacement([1, 2, 3], 2))
# [(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)]

# product – Cartesian product (nested loops)
list(itertools.product([1, 2], ['a', 'b']))
# [(1,'a'), (1,'b'), (2,'a'), (2,'b')]

# accumulate – running totals (prefix sums)
list(itertools.accumulate([1, 2, 3, 4]))
# [1, 3, 6, 10]

import operator
list(itertools.accumulate([3, 1, 4, 1], func=max))
# [3, 3, 4, 4] – running max

# chain – flatten multiple iterables
list(itertools.chain([1, 2], [3, 4], [5]))
# [1, 2, 3, 4, 5]

# chain.from_iterable – flatten list of lists
list(itertools.chain.from_iterable([[1, 2], [3, 4]]))
# [1, 2, 3, 4]

# groupby – group consecutive elements (must be sorted first!)
data = sorted([(1, 'a'), (2, 'b'), (1, 'c')], key=lambda x: x[0])
for key, group in itertools.groupby(data, key=lambda x: x[0]):
    print(key, list(group))
# 1 [(1, 'a'), (1, 'c')]
# 2 [(2, 'b')]

```

functools Module

```

from functools import lru_cache, reduce

# lru_cache – memoization decorator (CRITICAL for DP in interviews)
@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

fib(100) # Instant – O(n) with memoization vs O(2^n) without

# For 2D DP problems
@lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0:
        return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))

# reduce – fold a sequence
reduce(lambda a, b: a * b, [1, 2, 3, 4]) # 24 (product)
reduce(lambda a, b: a if a > b else b, [3, 1, 4]) # 4 (max)

```

math Module

```
import math

math.gcd(12, 8)           # 4
math.lcm(4, 6)           # 12 (Python 3.9+)
math.factorial(5)        # 120
math.inf                 # float('inf')
math.ceil(3.2)           # 4
math.floor(3.8)          # 3
math.sqrt(16)            # 4.0
math.isqrt(16)           # 4 (integer square root, Python 3.8+)
math.log(8, 2)           # 3.0 (log base 2 of 8)
math.log2(8)             # 3.0 (more precise)
math.comb(5, 2)          # 10 (C(5,2), Python 3.8+)
math.perm(5, 2)          # 20 (P(5,2), Python 3.8+)

# GCD of multiple numbers
from functools import reduce
reduce(math.gcd, [12, 8, 16]) # 4

# LCM of multiple numbers (Python 3.9+)
math.lcm(4, 6, 8)        # 24
```

string Module

```
import string

string.ascii_lowercase    # 'abcdefghijklmnopqrstuvwxyz'
string.ascii_uppercase    # 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
string.ascii_letters      # 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
string.digits             # '0123456789'
string.punctuation        # '!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~'

# Useful in interviews for character checks
def is_alphanumeric(c):
    return c in string.ascii_letters + string.digits
```

1.3 Pythonic Patterns

```

# enumerate – index + value
for i, val in enumerate(["a", "b", "c"]):
    print(i, val) # 0 a, 1 b, 2 c

for i, val in enumerate(arr, start=1): # custom start
    print(i, val)

# zip – iterate multiple sequences in parallel
names = ["Alice", "Bob"]
scores = [95, 87]
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# zip for creating dict
d = dict(zip(names, scores)) # {'Alice': 95, 'Bob': 87}

# zip for transposing a matrix
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposed = list(zip(*matrix)) # [(1,4,7), (2,5,8), (3,6,9)]

# Unpacking
a, b, *rest = [1, 2, 3, 4, 5] # a=1, b=2, rest=[3,4,5]
first, *middle, last = [1, 2, 3, 4] # first=1, middle=[2,3], last=4
a, b = b, a # swap

# Walrus operator (:=) – Python 3.8+
# Assign and use in same expression
while (line := input()) != "quit":
    process(line)

# Useful in list comprehensions
results = [y for x in data if (y := expensive(x)) > threshold]

# f-strings
name, age = "Alice", 30
f"{name} is {age} years old"
f"{3.14159:.2f}" # "3.14"
f"{1000000:,}" # "1,000,000"
f"{255:08b}" # "11111111" (binary, 8 digits)
f"{val!r}" # uses repr()

# List/Dict/Set comprehensions
squares = [x**2 for x in range(10)]
even_sq = [x**2 for x in range(10) if x % 2 == 0]
flat = [x for row in matrix for x in row] # flatten

freq = {k: v for k, v in counter.items() if v > 1}
seen = {x for x in arr if x > 0}

# Generator expressions – lazy, memory efficient
total = sum(x**2 for x in range(1000000)) # no list created
any(x > 100 for x in data) # short-circuits
all(x > 0 for x in data) # short-circuits

```

1.4 Common Pitfalls

Mutable Default Arguments

```
# BUG: default list is shared across calls
def append_to(element, lst=[]):
    lst.append(element)
    return lst

append_to(1) # [1]
append_to(2) # [1, 2] ← NOT [2]!

# FIX: use None as default
def append_to(element, lst=None):
    if lst is None:
        lst = []
    lst.append(element)
    return lst
```

Shallow vs Deep Copy

```
import copy

# Shallow copy – copies top-level only
a = [[1, 2], [3, 4]]
b = a.copy() # or list(a) or a[:]
b[0].append(5)
print(a) # [[1, 2, 5], [3, 4]] ← inner list was shared!

# Deep copy – copies everything recursively
c = copy.deepcopy(a)
c[0].append(6)
print(a) # [[1, 2, 5], [3, 4]] ← unaffected
```

Integer Caching

```
# CPython caches integers -5 to 256
a = 256
b = 256
a is b # True (same cached object)

a = 257
b = 257
a is b # False (different objects) – may vary!
```

is **VS** **==**

```
# is → identity (same object in memory)
# == → equality (same value)

a = [1, 2, 3]
b = [1, 2, 3]
a == b # True (same content)
a is b # False (different objects)

# Use 'is' only for: None, True, False
if x is None:
    pass
```

Pass by Object Reference

```
# Python uses "pass by object reference" (or "pass by assignment")
# Mutable objects CAN be modified in-place inside functions
# Immutable objects CANNOT – reassignment creates a new local reference

def modify(lst, num):
    lst.append(4) # modifies original list
    num += 1     # creates new local int – original unchanged

my_list = [1, 2, 3]
my_num = 10
modify(my_list, my_num)
print(my_list) # [1, 2, 3, 4] – modified!
print(my_num)  # 10 – unchanged!
```

1.5 OOP for Interviews

Dunder Methods

```
class Interval:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def __repr__(self):
        """For debugging – unambiguous representation"""
        return f"Interval({self.start}, {self.end})"

    def __str__(self):
        """For display – user-friendly"""
        return f"[{self.start}, {self.end})"

    def __eq__(self, other):
        """Enable == comparison"""
        return self.start == other.start and self.end == other.end

    def __hash__(self):
        """Required if __eq__ is defined and you want to use in sets/dicts"""
        return hash((self.start, self.end))

    def __lt__(self, other):
        """Enable sorting – sorted() and heapq use this"""
        if self.start == other.start:
            return self.end < other.end
        return self.start < other.start

    def __len__(self):
        return self.end - self.start

    def __contains__(self, point):
        """Enable 'in' operator: point in interval"""
        return self.start <= point <= self.end

# Usage
intervals = [Interval(3, 5), Interval(1, 4), Interval(1, 2)]
sorted(intervals) # Uses __lt__: [Interval(1, 2), Interval(1, 4), Interval(3, 5)]
```

@property **Decorator**

```

class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

    @property
    def area(self):
        return 3.14159 * self._radius ** 2

c = Circle(5)
c.area    # 78.53975 (no parentheses needed)
c.radius = -1 # raises ValueError

```

Abstract Classes

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h

    def perimeter(self):
        return 2 * (self.w + self.h)

# shape = Shape() # TypeError: Can't instantiate abstract class
rect = Rectangle(3, 4)
rect.area() # 12

```

1.6 Error Handling Patterns in Interviews

```

# EAFP (Easier to Ask Forgiveness than Permission) – Pythonic
try:
    value = my_dict[key]
except KeyError:
    value = default

# vs LBYL (Look Before You Leap) – less Pythonic
if key in my_dict:
    value = my_dict[key]
else:
    value = default

# Best: use dict.get()
value = my_dict.get(key, default)

# try/except/else/finally
try:
    result = risky_operation()
except ValueError as e:
    handle_error(e)
except (TypeError, KeyError):
    handle_other()
else:
    # runs only if NO exception
    process(result)
finally:
    # ALWAYS runs
    cleanup()

# Custom exceptions
class InvalidInputError(ValueError):
    pass

# Using raise from for exception chaining
try:
    x = int(user_input)
except ValueError as e:
    raise InvalidInputError(f"Bad input: {user_input}") from e

```

1.7 Big O Analysis

How to Calculate Time Complexity

Rules of thumb:

1. **Drop constants:** $O(2n) \rightarrow O(n)$
2. **Drop lower-order terms:** $O(n^2 + n) \rightarrow O(n^2)$
3. **Nested loops:** multiply — $O(n * m)$
4. **Sequential operations:** add — $O(n + m)$
5. **Recursion:** use recurrence relations or Master Theorem

Common complexities ranked:

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

For $n = 10^6$:

$O(1)$	→	1 operation	
$O(\log n)$	→	~20 operations	
$O(n)$	→	10^6 operations	(~1 sec)
$O(n \log n)$	→	$\sim 2 * 10^7$ operations	(~2 sec)
$O(n^2)$	→	10^{12} operations	(~hours) ← TOO SLOW

Interview rule: If $n \leq 10^7$, $O(n)$ is fine. If $n \leq 10^4$, $O(n^2)$ is fine. If $n \leq 20$, $O(2^n)$ is fine.

Examples

```
# O(1) – constant
def get_first(arr):
    return arr[0] if arr else None

# O(log n) – halving the search space
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# O(n) – single pass
def find_max(arr):
    result = float('-inf')
    for x in arr:
        result = max(result, x)
    return result

# O(n log n) – sorting
def has_duplicate(arr):
    arr.sort() # O(n log n)
    for i in range(1, len(arr)): # O(n)
        if arr[i] == arr[i-1]:
            return True
    return False

# O(n^2) – nested loops
def two_sum_brute(arr, target):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                return [i, j]

# O(2^n) – exponential recursion
def subsets(nums):
    if not nums:
        return [[]]
    rest = subsets(nums[1:])
    return rest + [[nums[0]] + s for s in rest]
```

Space Complexity

```

# O(1) space – in-place
def reverse_array(arr):
    l, r = 0, len(arr) - 1
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1
        r -= 1

# O(n) space – hash set / auxiliary array
def has_duplicate(arr):
    seen = set()
    for x in arr:
        if x in seen:
            return True
        seen.add(x)
    return False

# O(n) space – recursion call stack
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1) # n stack frames

```

Amortized Analysis

Amortized $O(1)$ means that while individual operations may occasionally be expensive (like resizing a dynamic array), the **average cost per operation** over a sequence of operations is $O(1)$.

Example: Dynamic Array (Python list)

- Most `append()` calls are $O(1)$ — just write to pre-allocated memory.
- Occasionally, the array is full and must be resized: allocate 2x memory, copy n elements $\rightarrow O(n)$.
- But this happens every $\sim n$ appends, so cost spread over n appends = $O(n)/n = O(1)$ amortized.

Formal method (Aggregate Analysis):

For n appends, total cost = n (normal appends) + $1 + 2 + 4 + \dots + n$ (copies during resizing) = $n + 2n = 3n \rightarrow O(n)$ total $\rightarrow O(1)$ amortized per operation.

Part 2: Data Structures — Deep Dive

2.1 Arrays (Python Lists)

What It Is

An **array** (Python `list`) is a contiguous, ordered collection of elements accessible by index. Think of it like a row of numbered mailboxes — you can instantly access any box by its number.

When to Use It

- Need $O(1)$ random access by index
- Need to store ordered data
- Iterating over all elements
- Problem involves subarrays, subsequences, or in-place swaps

Internal Working

Python lists are dynamic arrays backed by a C array of pointers. See Section 1.1 for details.

Visual Diagram

```
Index:  0      1      2      3      4
        +-----+-----+-----+-----+
        | 10  | 20  | 30  | 40  | 50  |
        +-----+-----+-----+-----+

Subarray [1:4]:
          +-----+-----+
          | 20  | 30  | 40  |
          +-----+-----+

Two Pointers (opposite direction):
  L →                                ← R
  ↓                                  ↓
+-----+-----+-----+-----+
| 10  | 20  | 30  | 40  | 50  |
+-----+-----+-----+-----+
```

Implementation

```

class DynamicArray:
    """Simplified dynamic array implementation."""
    def __init__(self):
        self._size = 0
        self._capacity = 1
        self._data = [None] * self._capacity

    def __len__(self):
        return self._size

    def __getitem__(self, index):
        if index < 0 or index >= self._size:
            raise IndexError("Index out of range")
        return self._data[index]

    def append(self, value):
        if self._size == self._capacity:
            self._resize(2 * self._capacity)
        self._data[self._size] = value
        self._size += 1

    def pop(self):
        if self._size == 0:
            raise IndexError("Pop from empty array")
        self._size -= 1
        val = self._data[self._size]
        self._data[self._size] = None
        if self._size < self._capacity // 4:
            self._resize(max(1, self._capacity // 2))
        return val

    def insert(self, index, value):
        if self._size == self._capacity:
            self._resize(2 * self._capacity)
        for i in range(self._size, index, -1):
            self._data[i] = self._data[i - 1]
        self._data[index] = value
        self._size += 1

    def _resize(self, new_cap):
        new_data = [None] * new_cap
        for i in range(self._size):
            new_data[i] = self._data[i]
        self._data = new_data
        self._capacity = new_cap

```

Operations & Complexity

Operation	Time	Space	Why
Access <code>arr[i]</code>	$O(1)$	$O(1)$	Direct pointer arithmetic
Append	$O(1)^*$	$O(1)$	Amortized — occasional $O(n)$ resize
Pop (end)	$O(1)$	$O(1)$	Just decrement size
Insert at index	$O(n)$	$O(1)$	Must shift subsequent elements
Delete at index	$O(n)$	$O(1)$	Must shift subsequent elements
Search	$O(n)$	$O(1)$	Linear scan
Sort	$O(n \log n)$	$O(n)$	Timsort
Slice <code>arr[i:j]</code>	$O(j-i)$	$O(j-i)$	Copies elements

Common Patterns

- **Two Pointers** (sorted arrays, partitioning)

- **Sliding Window** (subarray sums, substring problems)
- **Prefix Sums** (range sum queries)
- **Kadane's Algorithm** (maximum subarray)
- **Dutch National Flag** (3-way partition)

Edge Cases

- Empty array
- Single element
- All elements identical
- Already sorted / reverse sorted
- Integer overflow in sum calculations
- Negative numbers

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Two Sum (LC 1)	Easy	Hash Map	Store complement in dict
2	Best Time to Buy and Sell Stock (LC 121)	Easy	Kadane's	Track min price so far
3	Contains Duplicate (LC 217)	Easy	Hash Set	Set membership check
4	Product of Array Except Self (LC 238)	Medium	Prefix/Suffix	Left pass then right pass
5	Maximum Subarray (LC 53)	Medium	Kadane's	Reset running sum if negative
6	Merge Intervals (LC 56)	Medium	Sorting	Sort by start, merge overlapping
7	3Sum (LC 15)	Medium	Two Pointers	Sort + fix one + two pointer
8	Container With Most Water (LC 11)	Medium	Two Pointers	Move shorter side inward
9	Trapping Rain Water (LC 42)	Hard	Two Pointers/Stack	Min of left_max, right_max minus height
10	First Missing Positive (LC 41)	Hard	Cyclic Sort	Place each num at index num-1

2.2 Strings

What It Is

A **string** is an immutable sequence of characters. Think of it as a fixed message carved in stone — to change any part, you must create an entirely new stone.

When to Use It

- Text processing, pattern matching
- Palindrome, anagram, substring problems
- Encoding/decoding problems

Internal Working

Python strings are immutable arrays of Unicode code points (UTF-8 internally, but CPython may use Latin-1, UCS-

2, or UCS-4 depending on content). See Section 1.1 for immutability implications.

Visual Diagram

```
String: "HELLO"
Index:  0  1  2  3  4
      +---+---+---+---+
      | H | E | L | L | O |
      +---+---+---+---+
      -5 -4 -3 -2 -1      (negative indexing)

Palindrome check (two pointers):
  L →                ← R
  ↓                  ↓
+---+---+---+---+
| R | A | C | A | R |
+---+---+---+---+
s[L] == s[R] ? Yes → move inward
```

Key Operations & Complexity

Operation	Time	Notes
<code>s[i]</code>	$O(1)$	Direct access
<code>s + t</code>	$O(n+m)$	Creates new string
<code>s * k</code>	$O(n*k)$	Creates new string
<code>s.find(t)</code>	$O(n*m)$	Substring search
<code>s.split()</code>	$O(n)$	Returns list of parts
<code>''.join(list)</code>	$O(n)$	Efficient concatenation
<code>s[::-1]</code>	$O(n)$	Reverse (creates new string)
<code>s.lower()/upper()</code>	$O(n)$	Case conversion
<code>s.count(c)</code>	$O(n)$	Count occurrences

Common Patterns

- **Sliding Window** with character frequency maps
- **Two Pointers** for palindrome checks
- **Hash Map** for anagram/character counting
- **Trie** for prefix matching

Edge Cases

- Empty string
- Single character
- All same characters
- Unicode / special characters
- Spaces and whitespace handling
- Case sensitivity

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Valid Anagram (LC 242)	Easy	Counter	Compare character counts
2	Valid Palindrome (LC 125)	Easy	Two Pointers	Skip non-alphanumeric
3	Longest Substring Without Repeating (LC 3)	Medium	Sliding Window	Hash map for last seen index
4	Longest Palindromic Substring (LC 5)	Medium	Expand Around Center	Try each center point
5	Group Anagrams (LC 49)	Medium	Sorting/Hash	Sorted string as key
6	Minimum Window Substring (LC 76)	Hard	Sliding Window	Expand right, shrink left
7	Palindromic Substrings (LC 647)	Medium	Expand Around Center	Count all palindromes
8	Encode and Decode Strings (LC 271)	Medium	Delimiter	Length-prefix encoding

2.3 Linked Lists

What It Is

A **linked list** is a chain of nodes where each node holds a value and a pointer to the next node. Think of it like a scavenger hunt — each clue (node) tells you where to find the next one, but you can't skip ahead.

When to Use It

- Need $O(1)$ insertion/deletion at known position
- Don't need random access
- Implement stacks, queues, LRU caches
- Problems involving merging, reversing, cycle detection

Internal Working

Python doesn't have a built-in linked list. Each node is a separate heap-allocated object. Traversal requires following pointers — no cache-friendly contiguous memory.

Visual Diagram

```

Singly Linked List:
head
↓
[10|•]→[20|•]→[30|•]→[40|•]→ None

Doubly Linked List:
None ←[•|10|•]≠[•|20|•]≠[•|30|•]→ None
      head                tail

Cycle Detection (Floyd's):
      ┌───┐
      ↓   |
[1|•]→[2|•]→[3|•]→[4|•]→[5|•]
      ↑ slow      ↑ fast
  
```

Implementation

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class LinkedList:
    def __init__(self):
        self.head = None

    def prepend(self, val):
        """O(1) – insert at beginning."""
        self.head = ListNode(val, self.head)

    def append(self, val):
        """O(n) – insert at end (O(1) if we maintain a tail pointer)."""
        if not self.head:
            self.head = ListNode(val)
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = ListNode(val)

    def delete(self, val):
        """O(n) – delete first occurrence."""
        dummy = ListNode(0, self.head)
        prev, curr = dummy, self.head
        while curr:
            if curr.val == val:
                prev.next = curr.next
                break
            prev = curr
            curr = curr.next
        self.head = dummy.next

    def reverse(self):
        """O(n) time, O(1) space – reverse in place."""
        prev, curr = None, self.head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        self.head = prev

    def find_middle(self):
        """O(n) – slow/fast pointer technique."""
        slow = fast = self.head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow

    def has_cycle(self):
        """O(n) – Floyd's cycle detection."""
        slow = fast = self.head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:
                return True
        return False

    def to_list(self):
        result = []
        curr = self.head
        while curr:
            result.append(curr.val)
            curr = curr.next
        return result

```

Operations & Complexity

Operation	Time	Space	Why
Access by index	$O(n)$	$O(1)$	Must traverse from head
Prepend	$O(1)$	$O(1)$	Just update head pointer
Append (no tail)	$O(n)$	$O(1)$	Must traverse to end
Append (with tail)	$O(1)$	$O(1)$	Direct tail pointer access
Delete (given node)	$O(1)$	$O(1)$	Bypass the node
Search	$O(n)$	$O(1)$	Linear traversal
Reverse	$O(n)$	$O(1)$	Single pass with 3 pointers

Common Patterns

- **Dummy head node** — simplifies edge cases for head operations
- **Fast/Slow pointers** — find middle, detect cycle, find cycle start
- **Reverse** — iterative (3 pointers) or recursive
- **Merge** — merge two sorted lists (like merge sort merge step)

Edge Cases

- Empty list
- Single node
- Two nodes
- Cycle in list
- Operating on head or tail

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Reverse Linked List (LC 206)	Easy	Three Pointers	prev, curr, next
2	Merge Two Sorted Lists (LC 21)	Easy	Merge	Dummy head, compare
3	Linked List Cycle (LC 141)	Easy	Fast/Slow	Floyd's algorithm
4	Remove Nth Node From End (LC 19)	Medium	Two Pointers	Gap of n between pointers
5	Reorder List (LC 143)	Medium	Find Mid + Reverse + Merge	Three-step process
6	Copy List with Random Pointer (LC 138)	Medium	Hash Map	Old → new node mapping
7	Add Two Numbers (LC 2)	Medium	Simulation	Carry propagation
8	LRU Cache (LC 146)	Medium	Hash Map + DLL	$O(1)$ get and put
9	Reverse Nodes in k-Group (LC 25)	Hard	Reverse sublist	Count k, reverse, connect
10	Merge k Sorted Lists (LC 23)	Hard	Heap	Min-heap of list heads

2.4 Stacks

What It Is

A **stack** is a Last-In-First-Out (LIFO) data structure. Think of a stack of plates — you can only add or remove from the top.

When to Use It

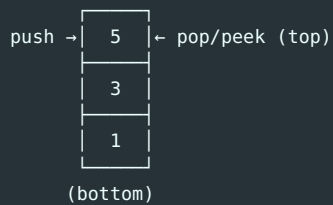
- Matching brackets/parentheses
- Processing nested structures (HTML, expressions)
- DFS traversal (explicit or implicit via recursion)
- Monotonic stack problems (next greater/smaller element)
- Undo operations, backtracking

Internal Working

In Python, use a `list` as a stack. `append()` and `pop()` are both $O(1)$ amortized and operate on the right end.

Visual Diagram

Stack (LIFO):



Matching parentheses:

Input: "{[()]}"

Stack states: (→ ({ → ([→ ({ → (→ empty ✓

Implementation

```

class Stack:
    def __init__(self):
        self._data = []

    def push(self, val):
        self._data.append(val)

    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self._data.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("Peek at empty stack")
        return self._data[-1]

    def is_empty(self):
        return len(self._data) == 0

    def __len__(self):
        return len(self._data)

# Min Stack — O(1) push, pop, and getMin
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        min_val = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(min_val)

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]

```

Operations & Complexity

Operation	Time	Space	Why
Push	$O(1)^*$	$O(1)$	Amortized — list append
Pop	$O(1)$	$O(1)$	Remove last element
Peek/Top	$O(1)$	$O(1)$	Access last element
isEmpty	$O(1)$	$O(1)$	Check length
Search	$O(n)$	$O(1)$	Must scan all elements

Common Patterns

- **Balanced parentheses** — push open, pop and match close
- **Monotonic stack** — maintain increasing/decreasing order for next greater/smaller element
- **Expression evaluation** — postfix, infix with operator precedence
- **Iterative DFS** — explicit stack replaces recursion

Edge Cases

- Empty stack operations
- Single element
- Nested structures of varying depth
- Stack overflow (recursion depth limit: `sys.setrecursionlimit()`)

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Valid Parentheses (LC 20)	Easy	Stack	Match open/close pairs
2	Min Stack (LC 155)	Medium	Auxiliary Stack	Track min alongside main stack
3	Evaluate Reverse Polish Notation (LC 150)	Medium	Stack	Operands push, operators pop 2
4	Daily Temperatures (LC 739)	Medium	Monotonic Stack	Decreasing stack, pop when warmer
5	Next Greater Element I (LC 496)	Easy	Monotonic Stack	Map from value to next greater
6	Largest Rectangle in Histogram (LC 84)	Hard	Monotonic Stack	Stack of increasing heights
7	Decode String (LC 394)	Medium	Stack	Push counts and partial strings
8	Basic Calculator (LC 224)	Hard	Stack	Handle signs and parentheses
9	Car Fleet (LC 853)	Medium	Stack	Sort by position, compare arrival
10	Asteroid Collision (LC 735)	Medium	Stack	Simulate collisions

2.5 Queues

What It Is

A **queue** is a First-In-First-Out (FIFO) data structure. Think of a line at a ticket counter — first person in line gets served first.

When to Use It

- BFS traversal
- Level-order tree traversal
- Task scheduling, buffer management
- Sliding window maximum (using deque)

Internal Working

Use `collections.deque` for O(1) operations on both ends. Do **not** use `list` as a queue — `list.pop(0)` is O(n).

Visual Diagram

Queue (FIFO):

enqueue (rear) dequeue (front)

↓ ↓

5	4	3	2	1
---	---	---	---	---

→ dequeue returns 1

Deque (double-ended):

appendleft append

↓ ↓

	←		→	
--	---	--	---	--

popleft pop

Implementation

```
from collections import deque

class Queue:
    def __init__(self):
        self._data = deque()

    def enqueue(self, val):
        self._data.append(val)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Dequeue from empty queue")
        return self._data.popleft()

    def peek(self):
        if self.is_empty():
            raise IndexError("Peek at empty queue")
        return self._data[0]

    def is_empty(self):
        return len(self._data) == 0

    def __len__(self):
        return len(self._data)

# Priority Queue using heapq
import heapq

class PriorityQueue:
    def __init__(self):
        self._heap = []
        self._counter = 0

    def push(self, priority, item):
        heapq.heappush(self._heap, (priority, self._counter, item))
        self._counter += 1

    def pop(self):
        return heapq.heappop(self._heap)[2]

    def peek(self):
        return self._heap[0][2]

    def is_empty(self):
        return len(self._heap) == 0
```

Operations & Complexity

Operation	Time (deque)	Time (list)	Notes
Enqueue (append)	$O(1)$	$O(1)$	Both efficient at end
Dequeue (popleft)	$O(1)$	$O(n)$	List shifts all elements!
Peek	$O(1)$	$O(1)$	Access first element
isEmpty	$O(1)$	$O(1)$	Check length

Edge Cases

- Empty queue operations
- Single element
- Circular queue boundary conditions

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Implement Queue using Stacks (LC 232)	Easy	Two Stacks	Lazy transfer on dequeue
2	Implement Stack using Queues (LC 225)	Easy	One Queue	Rotate n-1 elements
3	Number of Recent Calls (LC 933)	Easy	Queue	Remove old timestamps
4	Sliding Window Maximum (LC 239)	Hard	Monotonic Deque	Maintain decreasing deque
5	Design Circular Queue (LC 622)	Medium	Array	Front and rear pointers

2.6 Hash Maps (Dictionaries)

What It Is

A **hash map** stores key-value pairs with $O(1)$ average lookup. Think of it like a library catalog — given a book title (key), you instantly find its shelf location (value).

When to Use It

- Need $O(1)$ lookup, insert, delete
- Counting frequencies
- Finding pairs (two sum pattern)
- Caching / memoization
- Grouping elements

Internal Working

See Section 1.1 for Python dict internals (open-addressing hash table).

Visual Diagram

Key $\rightarrow$ hash() $\rightarrow$ bucket index $\rightarrow$ value

Bucket array:

Implementation

Operations & Complexity

Operation	Average	Worst	Why
Get	$O(1)$	$O(n)$	Hash lookup; worst = all collide
Put	$O(1)^*$	$O(n)$	Amortized due to resizing
Delete	$O(1)$	$O(n)$	Hash lookup + remove
Contains key	$O(1)$	$O(n)$	Hash lookup
Keys/Values	$O(n)$	$O(n)$	Iterate all entries

Common Patterns

- **Two Sum** — store complement → index mapping
- **Frequency counting** — Counter or defaultdict(int)
- **Grouping** — defaultdict(list) with common key
- **Prefix sum + hash map** — subarray sum equals k

Edge Cases

- Hash collisions
- Unhashable keys (lists, dicts — use tuple instead)
- `None` as key/value
- Modifying dict during iteration

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Two Sum (LC 1)	Easy	Hash Map	Complement lookup
2	Ransom Note (LC 383)	Easy	Counter	Character frequency
3	Group Anagrams (LC 49)	Medium	Hash + Sort	Sorted string as key
4	Top K Frequent Elements (LC 347)	Medium	Hash + Heap	Counter + nlargest
5	Subarray Sum Equals K (LC 560)	Medium	Prefix Sum + Hash	Map prefix_sum → count
6	Longest Consecutive Sequence (LC 128)	Medium	Hash Set	Check if num-1 not in set
7	Design HashMap (LC 706)	Easy	Implementation	Array of linked lists

2.7 Trees

What It Is

A **tree** is a hierarchical data structure with a root node and child nodes forming a parent-child relationship. Think of a family tree or a corporate org chart — each person has one boss (parent) but can manage multiple people (children).

When to Use It

- Hierarchical data (file systems, org charts)
- Sorted data with $O(\log n)$ operations (BST)

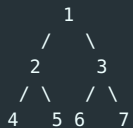
- Priority-based processing (heaps)
- Prefix matching (tries)

Internal Working

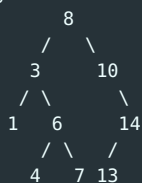
Each tree node is a heap-allocated Python object with `val`, `left`, `right` attributes. Trees are pointer-based — no contiguous memory.

Visual Diagram

Binary Tree:



Binary Search Tree (BST):



Level-order traversal (BFS):

Level 0: [8]
Level 1: [3, 10]
Level 2: [1, 6, 14]
Level 3: [4, 7, 13]

Implementation

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if not node:
            return TreeNode(val)
        if val < node.val:
            node.left = self._insert(node.left, val)
        elif val > node.val:
            node.right = self._insert(node.right, val)
        return node

    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if not node or node.val == val:
            return node
        if val < node.val:
            return self._search(node.left, val)
        return self._search(node.right, val)
```

```

def delete(self, val):
    self.root = self._delete(self.root, val)

def _delete(self, node, val):
    if not node:
        return None
    if val < node.val:
        node.left = self._delete(node.left, val)
    elif val > node.val:
        node.right = self._delete(node.right, val)
    else:
        if not node.left:
            return node.right
        if not node.right:
            return node.left
        # Find inorder successor (smallest in right subtree)
        successor = node.right
        while successor.left:
            successor = successor.left
        node.val = successor.val
        node.right = self._delete(node.right, successor.val)
    return node

# Tree Traversals
def inorder(root):
    """Left → Root → Right (gives sorted order for BST)"""
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

def preorder(root):
    """Root → Left → Right (useful for serialization)"""
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

def postorder(root):
    """Left → Right → Root (useful for deletion, expression evaluation)"""
    if not root:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]

def level_order(root):
    """BFS level by level"""
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result

# Iterative Inorder (using stack)
def inorder_iterative(root):
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result

# Tree Properties

```

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

def is_balanced(root):
    def check(node):
        if not node:
            return 0
        left = check(node.left)
        right = check(node.right)
        if left == -1 or right == -1 or abs(left - right) > 1:
            return -1
        return 1 + max(left, right)
    return check(root) != -1

def is_valid_bst(root, lo=float('-inf'), hi=float('inf')):
    if not root:
        return True
    if not (lo < root.val < hi):
        return False
    return (is_valid_bst(root.left, lo, root.val) and
            is_valid_bst(root.right, root.val, hi))
```

Operations & Complexity

Operation (BST)	Average	Worst (skewed)	Notes
Search	$O(\log n)$	$O(n)$	Balanced: $O(\log n)$
Insert	$O(\log n)$	$O(n)$	Degenerates to linked list
Delete	$O(\log n)$	$O(n)$	Find successor + reconnect
Inorder traversal	$O(n)$	$O(n)$	Visit all nodes
Height	$O(n)$	$O(n)$	DFS to all leaves

Common Patterns

- **DFS** (preorder, inorder, postorder) for most tree problems
- **BFS** for level-order, minimum depth
- **BST property** — inorder gives sorted order
- **Lowest Common Ancestor (LCA)** — recursive or with parent pointers
- **Path Sum** — DFS with running sum
- **Serialize/Deserialize** — preorder + null markers

Edge Cases

- Empty tree (root is None)
- Single node
- Skewed tree (all left or all right — degenerates to linked list)
- Negative values
- Duplicate values in BST

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Maximum Depth of Binary Tree (LC 104)	Easy	DFS	1 + max(left, right)
2	Invert Binary Tree (LC 226)	Easy	DFS	Swap children recursively
3	Same Tree (LC 100)	Easy	DFS	Compare recursively
4	Binary Tree Level Order Traversal (LC 102)	Medium	BFS	Queue + level size
5	Validate BST (LC 98)	Medium	DFS	Pass min/max bounds
6	Lowest Common Ancestor (LC 236)	Medium	DFS	Return when found
7	Binary Tree Right Side View (LC 199)	Medium	BFS	Last node per level
8	Construct from Preorder/Inorder (LC 105)	Medium	Recursion	Root splits inorder
9	Serialize/Deserialize (LC 297)	Hard	Preorder + Queue	Null markers
10	Binary Tree Maximum Path Sum (LC 124)	Hard	DFS	Track max through node

2.8 Heaps (Priority Queues)

What It Is

A **heap** is a complete binary tree where each parent is smaller (min-heap) or larger (max-heap) than its children. Think of it as a priority queue — the most important item is always at the front.

When to Use It

- Need quick access to min/max element
- Top-K problems
- Merge K sorted lists/streams
- Running median
- Dijkstra's shortest path
- Task scheduling by priority

Internal Working

Python's `heapq` uses a **list** as a min-heap. The tree is stored in array form:

- Parent of index `i` : `(i - 1) // 2`
- Left child: `2 * i + 1`
- Right child: `2 * i + 2`

Visual Diagram

Min-Heap as tree:	As array:
<pre> 1 /\ 3 2 /\ /\ 7 4 5 6 </pre>	<pre> [1, 3, 2, 7, 4, 5, 6] Index: 0 1 2 3 4 5 6 </pre>
	Parent of i : $(i-1)//2$ Left child: $2i+1$ Right child: $2i+2$

Heappush (insert 0):	
Step 1: Add at end	Step 2: Bubble up
<pre> 1 /\ 3 2 /\ /\ 7 4 5 0 </pre>	<pre> 0 /\ 1 2 /\ /\ 7 4 5 3 </pre>
↑ sift up	↑ done

Implementation

```

class MinHeap:
    def __init__(self):
        self.data = []

    def push(self, val):
        self.data.append(val)
        self._sift_up(len(self.data) - 1)

    def pop(self):
        if not self.data:
            raise IndexError("Pop from empty heap")
        self._swap(0, len(self.data) - 1)
        val = self.data.pop()
        if self.data:
            self._sift_down(0)
        return val

    def peek(self):
        return self.data[0]

    def _sift_up(self, i):
        while i > 0:
            parent = (i - 1) // 2
            if self.data[i] < self.data[parent]:
                self._swap(i, parent)
                i = parent
            else:
                break

    def _sift_down(self, i):
        n = len(self.data)
        while True:
            smallest = i
            left = 2 * i + 1
            right = 2 * i + 2
            if left < n and self.data[left] < self.data[smallest]:
                smallest = left
            if right < n and self.data[right] < self.data[smallest]:
                smallest = right
            if smallest != i:
                self._swap(i, smallest)
                i = smallest
            else:
                break

    def _swap(self, i, j):
        self.data[i], self.data[j] = self.data[j], self.data[i]

    def __len__(self):
        return len(self.data)

```

Operations & Complexity

Operation	Time	Space	Why
Push	$O(\log n)$	$O(1)$	Sift up at most tree height
Pop	$O(\log n)$	$O(1)$	Sift down at most tree height
Peek	$O(1)$	$O(1)$	Root is always min/max
Heapify	$O(n)$	$O(1)$	Bottom-up sift down (not $O(n \log n)$!)
Search	$O(n)$	$O(1)$	No ordering guarantee for non-root

Why heapify is $O(n)$: Most nodes are near the bottom (leaves need 0 sift-down), and very few nodes are near the top (root needs $\log n$ sift-down). Summing over all levels: $n/2 * 0 + n/4 * 1 + n/8 * 2 + \dots = O(n)$.

Edge Cases

- Single element
- Duplicate priorities
- Max-heap simulation (negate values)
- Custom comparison (use tuples or `__lt__`)

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Kth Largest Element (LC 215)	Medium	Min-Heap size k	Push, pop if > k
2	Top K Frequent Elements (LC 347)	Medium	Counter + Heap	nlargest or bucket sort
3	Merge k Sorted Lists (LC 23)	Hard	Min-Heap	Heap of (val, index, node)
4	Find Median from Data Stream (LC 295)	Hard	Two Heaps	Max-heap (small) + min-heap (large)
5	Task Scheduler (LC 621)	Medium	Greedy + Heap	Max-freq tasks first
6	K Closest Points to Origin (LC 973)	Medium	Max-Heap size k	Distance as priority
7	Reorganize String (LC 767)	Medium	Greedy + Heap	Place most frequent first
8	Kth Smallest in Sorted Matrix (LC 378)	Medium	Min-Heap	Push first col, expand right

2.9 Graphs

What It Is

A **graph** is a collection of **nodes** (vertices) connected by **edges**. Think of it like a social network — people are nodes, and friendships are edges.

When to Use It

- Network/relationship modeling
- Shortest path problems
- Connectivity (is A reachable from B?)
- Cycle detection
- Topological ordering (task dependencies)
- Grid/matrix traversal (treat each cell as a node)

Internal Working

Python graphs are typically represented as adjacency lists using `defaultdict(list)` or `dict` . Adjacency matrices (`list[list]`) are used for dense graphs.

Visual Diagram

```

Undirected Graph:          Directed Graph (DAG):
  1 --- 2                  1 → 2
  |   / |                  ↓   ↓
  |   / |                  3 → 4
  3 --- 4                  ↓
                              5

```

```

Adjacency List (undirected):  Adjacency Matrix:
1: [2, 3]                   | 1 2 3 4
2: [1, 3, 4]                +-----+
3: [1, 2, 4]                1| 0 1 1 0
4: [2, 3]                   2| 1 0 1 1
                              3| 1 1 0 1
                              4| 0 1 1 0

```

Grid as implicit graph:
Each cell (r, c) connects to 4 neighbors:
(r-1,c), (r+1,c), (r,c-1), (r,c+1)

Implementation

```

from collections import defaultdict, deque

class Graph:
    def __init__(self, directed=False):
        self.adj = defaultdict(list)
        self.directed = directed

    def add_edge(self, u, v, weight=1):
        self.adj[u].append((v, weight))
        if not self.directed:
            self.adj[v].append((u, weight))

    def bfs(self, start):
        """O(V + E) - Breadth-First Search."""
        visited = {start}
        queue = deque([start])
        order = []
        while queue:
            node = queue.popleft()
            order.append(node)
            for neighbor, _ in self.adj[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)
        return order

    def dfs(self, start):
        """O(V + E) - Depth-First Search (iterative)."""
        visited = set()
        stack = [start]
        order = []
        while stack:
            node = stack.pop()
            if node in visited:
                continue
            visited.add(node)
            order.append(node)
            for neighbor, _ in self.adj[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
        return order

    def dfs_recursive(self, start, visited=None):
        if visited is None:
            visited = set()
        visited.add(start)
        result = [start]
        for neighbor, _ in self.adj[start]:
            if neighbor not in visited:

```

```

        result.extend(self.dfs_recursive(neighbor, visited))
    return result

def has_cycle_undirected(self):
    """Detect cycle in undirected graph using DFS."""
    visited = set()
    for node in self.adj:
        if node not in visited:
            if self._dfs_cycle(node, -1, visited):
                return True
    return False

def _dfs_cycle(self, node, parent, visited):
    visited.add(node)
    for neighbor, _ in self.adj[node]:
        if neighbor not in visited:
            if self._dfs_cycle(neighbor, node, visited):
                return True
        elif neighbor != parent:
            return True
    return False

def has_cycle_directed(self):
    """Detect cycle in directed graph using coloring."""
    WHITE, GRAY, BLACK = 0, 1, 2
    color = defaultdict(int)

    def dfs(node):
        color[node] = GRAY
        for neighbor, _ in self.adj[node]:
            if color[neighbor] == GRAY:
                return True
            if color[neighbor] == WHITE and dfs(neighbor):
                return True
        color[node] = BLACK
        return False

    return any(color[node] == WHITE and dfs(node) for node in self.adj)

def shortest_path_bfs(self, start, end):
    """BFS shortest path (unweighted graph)."""
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == end:
            return path
        for neighbor, _ in self.adj[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None

def topological_sort(self):
    """Kahn's algorithm (BFS-based)."""
    in_degree = defaultdict(int)
    for node in self.adj:
        for neighbor, _ in self.adj[node]:
            in_degree[neighbor] += 1

    queue = deque([n for n in self.adj if in_degree[n] == 0])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor, _ in self.adj[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(order) != len(self.adj):
        return None # cycle detected
    return order

```

Operations & Complexity

Operation	Adj List	Adj Matrix	Notes
Add edge	$O(1)$	$O(1)$	Append to list / set cell
Remove edge	$O(E)$	$O(1)$	Search list vs set cell
Check if edge exists	$O(\text{degree})$	$O(1)$	Scan neighbors vs array
Get all neighbors	$O(\text{degree})$	$O(V)$	Return list vs scan row
BFS / DFS	$O(V + E)$	$O(V^2)$	Visit all vertices + edges
Space	$O(V + E)$	$O(V^2)$	List vs matrix

Common Patterns

- **BFS** — shortest path (unweighted), level-order, multi-source BFS
- **DFS** — connectivity, cycle detection, topological sort, backtracking
- **Topological Sort** — task ordering, course prerequisites
- **Union-Find** — connected components, cycle detection (undirected)
- **Dijkstra/Bellman-Ford** — weighted shortest paths
- **Grid traversal** — flood fill, number of islands, shortest path in maze

Edge Cases

- Disconnected graph
- Self-loops
- Parallel edges
- Empty graph
- Single node

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Number of Islands (LC 200)	Medium	BFS/DFS on Grid	Flood fill from each '1'
2	Clone Graph (LC 133)	Medium	BFS/DFS + Hash	Map old→new nodes
3	Course Schedule (LC 207)	Medium	Topological Sort	Detect cycle in DAG
4	Course Schedule II (LC 210)	Medium	Topological Sort	Kahn's algorithm
5	Pacific Atlantic Water Flow (LC 417)	Medium	Multi-source BFS	BFS from each ocean
6	Number of Connected Components (LC 323)	Medium	Union-Find/DFS	Count components
7	Graph Valid Tree (LC 261)	Medium	Union-Find	$V-1$ edges + connected
8	Word Ladder (LC 127)	Hard	BFS	Shortest transformation
9	Alien Dictionary (LC 269)	Hard	Topological Sort	Build graph from word order
10	Network Delay Time (LC 743)	Medium	Dijkstra	Weighted shortest path
11	Cheapest Flights Within K Stops (LC 787)	Medium	BFS/Bellman-Ford	Modified BFS with k limit
12	Surrounded Regions (LC 130)	Medium	BFS/DFS	Start from border O's

2.10 Tries (Prefix Trees)

What It Is

A **trie** is a tree-like data structure for storing strings where each node represents a character. Think of it like an autocomplete system — typing "app" leads you down a path that branches to "apple", "application", "append", etc.

When to Use It

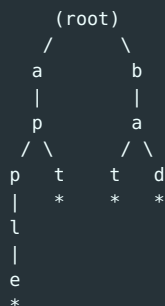
- Prefix-based search (autocomplete)
- Word dictionary with efficient lookup
- Spell checker
- IP routing (longest prefix match)
- Word search in grid

Internal Working

Each node contains a dictionary of children (char → node) and a boolean flag indicating if it's the end of a word. Space usage depends on the alphabet and shared prefixes.

Visual Diagram

Trie storing: ["apple", "app", "apt", "bat", "bad"]



* = end of word

Shared prefix "ap" saves space vs storing separately

Implementation

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """O(m) where m = len(word)."""
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    def search(self, word):
        """O(m) - exact match."""
        node = self._find_node(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        """O(m) - prefix match."""
        return self._find_node(prefix) is not None

    def _find_node(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node.children:
                return None
            node = node.children[char]
        return node

    def autocomplete(self, prefix):
        """Return all words with given prefix."""
        node = self._find_node(prefix)
        if not node:
            return []
        results = []
        self._dfs_collect(node, prefix, results)
        return results

    def _dfs_collect(self, node, path, results):
        if node.is_end:
            results.append(path)
        for char, child in node.children.items():
            self._dfs_collect(child, path + char, results)

    def delete(self, word):
        """Delete word from trie."""
        self._delete(self.root, word, 0)

    def _delete(self, node, word, depth):
        if depth == len(word):
            if not node.is_end:
                return False
            node.is_end = False
            return len(node.children) == 0
        char = word[depth]
        if char not in node.children:
            return False
        should_delete = self._delete(node.children[char], word, depth + 1)
        if should_delete:
            del node.children[char]
            return not node.is_end and len(node.children) == 0
        return False

```

Operations & Complexity

Operation	Time	Space	Notes
Insert	$O(m)$	$O(m)$	m = word length
Search	$O(m)$	$O(1)$	Follow pointers
Starts with	$O(m)$	$O(1)$	Same as search without end check
Delete	$O(m)$	$O(1)$	May remove empty branches
Space total	—	$O(N*m)$	N words, m avg length (worst)

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Implement Trie (LC 208)	Medium	Basic Trie	Dict of children + is_end
2	Design Add and Search Words (LC 211)	Medium	Trie + DFS	'.' matches any char
3	Word Search II (LC 212)	Hard	Trie + Backtracking	Build trie from words, DFS on grid
4	Replace Words (LC 648)	Medium	Trie	Find shortest prefix match
5	Search Suggestions System (LC 1268)	Medium	Trie + DFS	Autocomplete with limit 3

2.11 Union-Find (Disjoint Set)

What It Is

Union-Find is a data structure that tracks elements partitioned into disjoint (non-overlapping) sets. Think of it like country borders — you can merge countries (union) and check if two cities are in the same country (find).

When to Use It

- Connected components in undirected graphs
- Cycle detection in undirected graphs
- Kruskal's MST algorithm
- Dynamic connectivity (online queries)
- Redundant connection problems

Internal Working

Each element points to a parent. Initially, each element is its own parent (self-loop). **Path compression** flattens the tree on `find()`, and **union by rank** keeps trees balanced.

Visual Diagram

Initial state (5 elements, each its own set):
[0] [1] [2] [3] [4]

After union(0,1), union(2,3), union(3,4):

```
  0       2
  |       / \
  1      3  4
```

After union(0,2):

```
  0
  / \
 1  2
  / \
 3  4
```

After find(4) with path compression:

```
  0
  / | \
 1 2  4
    |
    3
```

(4 now points directly to root 0)

Implementation

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        """Find root with path compression – near O(1) amortized."""
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        """Union by rank – keeps tree balanced."""
        px, py = self.find(x), self.find(y)
        if px == py:
            return False # already in same set
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        self.components -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)

    def get_components(self):
        return self.components
```

Operations & Complexity

Operation	Time	Notes
Find	$O(\alpha(n)) \approx O(1)$	α = inverse Ackermann, practically constant
Union	$O(\alpha(n)) \approx O(1)$	Path compression + union by rank
Connected	$O(\alpha(n)) \approx O(1)$	Two find operations
Space	$O(n)$	Parent + rank arrays

Curated Problems

#	Problem	Difficulty	Pattern	Hint
1	Number of Connected Components (LC 323)	Medium	Basic UF	Union all edges, count components
2	Redundant Connection (LC 684)	Medium	Cycle Detection	Edge that creates cycle
3	Graph Valid Tree (LC 261)	Medium	UF	n-1 edges + 1 component
4	Accounts Merge (LC 721)	Medium	UF + Hash	Union accounts with shared emails
5	Longest Consecutive Sequence (LC 128)	Medium	UF	Union consecutive numbers

2.12 Monotonic Stack / Queue

What It Is

A **monotonic stack** maintains elements in strictly increasing or decreasing order. When a new element violates the monotonic property, elements are popped until the property is restored.

When to Use It

- Next Greater / Smaller Element
- Previous Greater / Smaller Element
- Largest Rectangle in Histogram
- Stock span problems
- Sliding window maximum (monotonic deque)

Pattern Explanation

Finding Next Greater Element using Monotonic Decreasing Stack:

Array: [2, 1, 2, 4, 3]

Step-by-step:

```
i=0: stack=[], push 0      stack=[0(2)]
i=1: 1 < 2, push 1         stack=[0(2), 1(1)]
i=2: 2 > 1, pop 1→nge[1]=2  stack=[0(2)]
      2 = 2, push 2         stack=[0(2), 2(2)]
i=3: 4 > 2, pop 2→nge[2]=4  stack=[0(2)]
      4 > 2, pop 0→nge[0]=4  stack=[]
      push 3                stack=[3(4)]
i=4: 3 < 4, push 4         stack=[3(4), 4(3)]
Remaining: nge[3]=-1, nge[4]=-1
```

Result: [4, 2, 4, -1, -1]

Template Code

```

# Next Greater Element (to the right)
def next_greater_element(nums):
    n = len(nums)
    result = [-1] * n
    stack = [] # stores indices, maintaining decreasing values
    for i in range(n):
        while stack and nums[i] > nums[stack[-1]]:
            result[stack.pop()] = nums[i]
        stack.append(i)
    return result

# Next Smaller Element (to the right)
def next_smaller_element(nums):
    n = len(nums)
    result = [-1] * n
    stack = [] # stores indices, maintaining increasing values
    for i in range(n):
        while stack and nums[i] < nums[stack[-1]]:
            result[stack.pop()] = nums[i]
        stack.append(i)
    return result

# Sliding Window Maximum using Monotonic Deque
from collections import deque

def max_sliding_window(nums, k):
    dq = deque() # stores indices, maintaining decreasing values
    result = []
    for i in range(len(nums)):
        # Remove indices outside window
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        # Maintain decreasing order
        while dq and nums[dq[-1]] <= nums[i]:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

```

2.13 Segment Tree / Binary Indexed Tree (Advanced)

Segment Tree

A **segment tree** is a binary tree where each node represents an interval (segment) of the array. It supports **range queries** (sum, min, max) and **point/range updates** in $O(\log n)$.

```

class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.tree = [0] * (4 * self.n)
        self._build(nums, 0, 0, self.n - 1)

    def _build(self, nums, node, start, end):
        if start == end:
            self.tree[node] = nums[start]
            return
        mid = (start + end) // 2
        self._build(nums, 2 * node + 1, start, mid)
        self._build(nums, 2 * node + 2, mid + 1, end)
        self.tree[node] = self.tree[2 * node + 1] + self.tree[2 * node + 2]

    def update(self, idx, val, node=0, start=0, end=None):
        if end is None:
            end = self.n - 1
        if start == end:
            self.tree[node] = val
            return
        mid = (start + end) // 2
        if idx <= mid:
            self.update(idx, val, 2 * node + 1, start, mid)
        else:
            self.update(idx, val, 2 * node + 2, mid + 1, end)
        self.tree[node] = self.tree[2 * node + 1] + self.tree[2 * node + 2]

    def query(self, l, r, node=0, start=0, end=None):
        """Range sum query [l, r]."""
        if end is None:
            end = self.n - 1
        if r < start or end < l:
            return 0
        if l <= start and end <= r:
            return self.tree[node]
        mid = (start + end) // 2
        return (self.query(l, r, 2 * node + 1, start, mid) +
                self.query(l, r, 2 * node + 2, mid + 1, end))

```

Binary Indexed Tree (Fenwick Tree)

A more compact alternative for prefix sum queries and point updates.

```

class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)

    def update(self, i, delta):
        """Add delta to index i (1-indexed)."""
        i += 1
        while i <= self.n:
            self.tree[i] += delta
            i += i & (-i)

    def query(self, i):
        """Prefix sum [0, i] (0-indexed)."""
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & (-i)
        return total

    def range_query(self, l, r):
        """Range sum [l, r] (0-indexed)."""
        return self.query(r) - (self.query(l - 1) if l > 0 else 0)

```

Structure	Build	Point Update	Range Query	Space
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(4n)$
BIT/Fenwick	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$

Part 3: Algorithm Patterns — The Core of Interview Prep

3.1 Two Pointers

Explanation

Use two pointers to traverse a data structure (usually from different ends or at different speeds) to reduce brute-force $O(n^2)$ to $O(n)$.

When to Use

- Sorted array / linked list
- Pair/triplet with target sum
- Removing duplicates
- Partitioning (Dutch National Flag)
- Palindrome checks

Variants

1. **Opposite direction** — one from start, one from end
2. **Same direction** — both start from beginning (fast/slow)
3. **Fast/Slow** — cycle detection, middle finding

Template Code

```

# Opposite Direction – Two Sum II (sorted array)
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        elif total < target:
            left += 1
        else:
            right -= 1
    return []

# Same Direction – Remove Duplicates from Sorted Array
def remove_duplicates(nums):
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1

# Fast/Slow – Linked List Cycle Start
def detect_cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        slow = head
        while slow is not fast:
            slow = slow.next
            fast = fast.next
        return slow
    return None

```

Dry Run

Two Sum Sorted: nums = [2, 7, 11, 15], target = 9

```

left=0, right=3: 2+15=17 > 9 → right=2
left=0, right=2: 2+11=13 > 9 → right=1
left=0, right=1: 2+7=9 == 9 → return [0, 1] ✓

```

Curated Problems

#	Problem	Difficulty	Variant
1	Two Sum II (LC 167)	Medium	Opposite
2	3Sum (LC 15)	Medium	Opposite + fix one
3	Container With Most Water (LC 11)	Medium	Opposite
4	Trapping Rain Water (LC 42)	Hard	Opposite
5	Remove Duplicates (LC 26)	Easy	Same direction
6	Move Zeroes (LC 283)	Easy	Same direction
7	Linked List Cycle II (LC 142)	Medium	Fast/Slow
8	Sort Colors (LC 75)	Medium	Three pointers
9	Palindrome Linked List (LC 234)	Easy	Fast/Slow + reverse
10	Squares of Sorted Array (LC 977)	Easy	Opposite

3.2 Sliding Window

Explanation

Maintain a window (subarray/substring) that slides over the data. Instead of recalculating from scratch, update the window by adding/removing elements at the edges.

When to Use

- Subarray/substring with some constraint (sum, distinct chars, etc.)
- Problem asks for "maximum/minimum length subarray/substring"
- Contiguous elements

Variants

1. **Fixed size** — window of exactly k elements
2. **Variable size** — expand right until constraint violated, shrink left
3. **With hash map** — track character/element frequencies

Template Code

```

# Fixed-size window - Maximum Sum Subarray of Size K
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])
    max_sum = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum

# Variable-size window - Minimum Size Subarray Sum
def min_subarray_len(target, nums):
    left = 0
    current_sum = 0
    min_len = float('inf')
    for right in range(len(nums)):
        current_sum += nums[right]
        while current_sum >= target:
            min_len = min(min_len, right - left + 1)
            current_sum -= nums[left]
            left += 1
    return min_len if min_len != float('inf') else 0

# With Hash Map - Longest Substring Without Repeating Characters
def length_of_longest_substring(s):
    char_index = {}
    left = 0
    max_len = 0
    for right in range(len(s)):
        if s[right] in char_index and char_index[s[right]] >= left:
            left = char_index[s[right]] + 1
        char_index[s[right]] = right
        max_len = max(max_len, right - left + 1)
    return max_len

# Minimum Window Substring (hard classic)
from collections import Counter

def min_window(s, t):
    need = Counter(t)
    missing = len(t)
    left = 0
    start, end = 0, float('inf')
    for right, char in enumerate(s):
        if need[char] > 0:
            missing -= 1
            need[char] -= 1
        while missing == 0:
            if right - left < end - start:
                start, end = left, right
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return s[start:end + 1] if end != float('inf') else ""

```

Dry Run

```
Longest Substring Without Repeating Characters: s = "abcabcbb"
```

```
right=0 'a': char_index={'a':0}, left=0, len=1
right=1 'b': char_index={'a':0,'b':1}, left=0, len=2
right=2 'c': char_index={'a':0,'b':1,'c':2}, left=0, len=3
right=3 'a': 'a' at 0 >= left(0), left=1
      char_index={'a':3,'b':1,'c':2}, len=3
right=4 'b': 'b' at 1 >= left(1), left=2
      char_index={'a':3,'b':4,'c':2}, len=3
right=5 'c': 'c' at 2 >= left(2), left=3
      char_index={'a':3,'b':4,'c':5}, len=3
right=6 'b': 'b' at 4 >= left(3), left=5, len=2
right=7 'b': 'b' at 6 >= left(5), left=7, len=1
```

```
Answer: 3 ("abc")
```

Curated Problems

#	Problem	Difficulty	Variant
1	Maximum Average Subarray I (LC 643)	Easy	Fixed
2	Longest Substring Without Repeating (LC 3)	Medium	Variable + Hash
3	Minimum Size Subarray Sum (LC 209)	Medium	Variable
4	Permutation in String (LC 567)	Medium	Fixed + Hash
5	Fruit Into Baskets (LC 904)	Medium	Variable + Hash
6	Longest Repeating Character Replacement (LC 424)	Medium	Variable
7	Minimum Window Substring (LC 76)	Hard	Variable + Hash
8	Sliding Window Maximum (LC 239)	Hard	Monotonic Deque
9	Substring with Concatenation (LC 30)	Hard	Fixed + Hash
10	Max Consecutive Ones III (LC 1004)	Medium	Variable

3.3 Binary Search

Explanation

Repeatedly halve the search space by comparing with the middle element. Works on sorted data or when the problem has a monotonic property (binary search on answer space).

When to Use

- **Sorted array** — find element, boundary, insert position
- **Rotated sorted array** — find minimum, search element
- **Answer space** — minimize maximum, capacity problems (Koko eating bananas)
- **2D matrix** — treat as 1D or search rows/columns

Template Code

```
# Standard Binary Search
def binary_search(nums, target):
    lo, hi = 0, len(nums) - 1
```

```

while lo <= hi:
    mid = lo + (hi - lo) // 2 # avoid overflow
    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        lo = mid + 1
    else:
        hi = mid - 1
return -1

# Find Left Boundary (first occurrence)
def bisect_left_manual(nums, target):
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo

# Find Right Boundary (last occurrence)
def bisect_right_manual(nums, target):
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] <= target:
            lo = mid + 1
        else:
            hi = mid
    return lo

# Binary Search on Answer Space – Koko Eating Bananas
def min_eating_speed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        hours = sum((p + mid - 1) // mid for p in piles) # ceil division
        if hours <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo

# Search in Rotated Sorted Array
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        # Left half is sorted
        if nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        # Right half is sorted
        else:
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

# Search 2D Matrix (rows sorted, first of next row > last of prev row)
def search_matrix(matrix, target):
    if not matrix:
        return False
    m, n = len(matrix), len(matrix[0])
    lo, hi = 0, m * n - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        val = matrix[mid // n][mid % n]

```

```

    if val == target:
        return True
    elif val < target:
        lo = mid + 1
    else:
        hi = mid - 1
    return False

```

Dry Run

Binary Search on Answer Space: piles = [3, 6, 7, 11], h = 8

```

lo=1, hi=11
mid=6: hours = ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6) = 1+1+2+2 = 6 ≤ 8 → hi=6
mid=3: hours = 1+2+3+4 = 10 > 8 → lo=4
mid=5: hours = 1+2+2+3 = 8 ≤ 8 → hi=5
mid=4: hours = 1+2+2+3 = 8 ≤ 8 → hi=4
lo=4, hi=4 → return 4 ✓

```

Curated Problems

#	Problem	Difficulty	Variant
1	Binary Search (LC 704)	Easy	Standard
2	Search Insert Position (LC 35)	Easy	Left boundary
3	Find First and Last Position (LC 34)	Medium	Both boundaries
4	Search in Rotated Sorted Array (LC 33)	Medium	Rotated
5	Find Minimum in Rotated Sorted Array (LC 153)	Medium	Rotated
6	Koko Eating Bananas (LC 875)	Medium	Answer space
7	Search a 2D Matrix (LC 74)	Medium	2D
8	Time Based Key-Value Store (LC 981)	Medium	Upper bound
9	Median of Two Sorted Arrays (LC 4)	Hard	Partition
10	Split Array Largest Sum (LC 410)	Hard	Answer space

3.4 BFS (Breadth-First Search)

Explanation

Explore all nodes at the current depth before moving to nodes at the next depth. Uses a queue. Guarantees shortest path in unweighted graphs.

When to Use

- **Shortest path** in unweighted graph
- **Level-order traversal** in trees
- **Multi-source BFS** — start from multiple sources simultaneously
- Grid problems asking for minimum steps

Template Code

```

from collections import deque

# Standard BFS – Shortest Path
def bfs_shortest_path(graph, start, end):
    queue = deque([(start, 0)]) # (node, distance)
    visited = {start}
    while queue:
        node, dist = queue.popleft()
        if node == end:
            return dist
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))
    return -1

# Level Order Traversal
def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result

# Multi-Source BFS – Rotten Oranges
def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1
    if fresh == 0:
        return 0
    max_time = 0
    while queue:
        r, c, time = queue.popleft()
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, time + 1))
                max_time = max(max_time, time + 1)
    return max_time if fresh == 0 else -1

# BFS on Grid – Shortest Path in Binary Matrix
def shortest_path_binary_matrix(grid):
    n = len(grid)
    if grid[0][0] == 1 or grid[n-1][n-1] == 1:
        return -1
    queue = deque([(0, 0, 1)])
    grid[0][0] = 1 # mark visited
    directions = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
    while queue:
        r, c, dist = queue.popleft()
        if r == n - 1 and c == n - 1:
            return dist
        for dr, dc in directions:
            nr, nc = r + dr, c + dc

```

```

        if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
            grid[nr][nc] = 1
            queue.append((nr, nc, dist + 1))
    return -1

```

Dry Run

BFS Level Order:

```

      1
     /\
    2  3
   /\
  4  5

```

Queue states:

```

[1]      → level 0: [1]
[2, 3]   → level 1: [2, 3]
[4, 5]   → level 2: [4, 5]
[]        → done

```

Result: [[1], [2, 3], [4, 5]]

Curated Problems

#	Problem	Difficulty	Variant
1	Binary Tree Level Order (LC 102)	Medium	Tree BFS
2	Rotting Oranges (LC 994)	Medium	Multi-source
3	Word Ladder (LC 127)	Hard	Shortest path
4	Shortest Path in Binary Matrix (LC 1091)	Medium	Grid BFS
5	Open the Lock (LC 752)	Medium	State BFS

3.5 DFS (Depth-First Search)

Explanation

Explore as deep as possible before backtracking. Uses recursion (implicit stack) or an explicit stack.

When to Use

- **Tree traversal** (preorder, inorder, postorder)
- **Graph connectivity** (is there a path?)
- **Cycle detection**
- **Grid traversal** (islands, flood fill)
- **Backtracking** (subsets, permutations)

Template Code

```

# DFS on Tree – Path Sum
def has_path_sum(root, target_sum):
    if not root:
        return False
    if not root.left and not root.right:
        return root.val == target_sum
    return (has_path_sum(root.left, target_sum - root.val) or
            has_path_sum(root.right, target_sum - root.val))

# DFS on Graph – Connected Components
def count_components(n, edges):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited = set()
    count = 0
    for i in range(n):
        if i not in visited:
            dfs(graph, i, visited)
            count += 1
    return count

def dfs(graph, node, visited):
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

# DFS on Grid – Number of Islands
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                dfs_grid(grid, r, c)
                count += 1
    return count

def dfs_grid(grid, r, c):
    if (r < 0 or r >= len(grid) or c < 0 or c >= len(grid[0])
        or grid[r][c] != '1'):
        return
    grid[r][c] = '0' # mark visited
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        dfs_grid(grid, r + dr, c + dc)

# Iterative DFS
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    result = []
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                stack.append(neighbor)
    return result

```

Curated Problems

#	Problem	Difficulty	Variant
1	Number of Islands (LC 200)	Medium	Grid DFS
2	Max Area of Island (LC 695)	Medium	Grid DFS
3	Clone Graph (LC 133)	Medium	Graph DFS
4	Path Sum (LC 112)	Easy	Tree DFS
5	Path Sum II (LC 113)	Medium	Tree DFS + backtracking

3.6 Backtracking

Explanation

Systematically explore all possible solutions by making choices, exploring consequences, and **undoing** (backtracking) choices that don't lead to valid solutions.

When to Use

- Generate all subsets, permutations, combinations
- Constraint satisfaction (Sudoku, N-Queens)
- Word search in grid
- Problems with "find all" or "generate all"

Template Code

```
# Subsets
def subsets(nums):
    result = []
    def backtrack(start, current):
        result.append(current[:]) # copy
        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1, current)
            current.pop() # undo choice
    backtrack(0, [])
    return result

# Permutations
def permutations(nums):
    result = []
    def backtrack(current, remaining):
        if not remaining:
            result.append(current[:])
            return
        for i in range(len(remaining)):
            current.append(remaining[i])
            backtrack(current, remaining[:i] + remaining[i+1:])
            current.pop()
    backtrack([], nums)
    return result

# Combinations (C(n, k))
def combinations(n, k):
    result = []
    def backtrack(start, current):
        if len(current) == k:
            result.append(current[:])
            return
        for i in range(start, n + 1):
```

```

        current.append(i)
        backtrack(i + 1, current)
        current.pop()
    backtrack(1, [])
    return result

# Combination Sum (can reuse elements)
def combination_sum(candidates, target):
    result = []
    def backtrack(start, current, remaining):
        if remaining == 0:
            result.append(current[:])
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            current.append(candidates[i])
            backtrack(i, current, remaining - candidates[i]) # i not i+1: reuse
            current.pop()
    backtrack(0, [], target)
    return result

# N-Queens
def solve_n_queens(n):
    result = []
    board = [['.' ] * n for _ in range(n)]
    cols = set()
    diag1 = set() # row - col
    diag2 = set() # row + col

    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            board[row][col] = 'Q'
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            backtrack(row + 1)
            board[row][col] = '.'
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)
    return result

```

Dry Run

Subsets of [1, 2, 3]:

```
backtrack(0, [])
→ add []
i=0: backtrack(1, [1])
→ add [1]
i=1: backtrack(2, [1,2])
→ add [1,2]
i=2: backtrack(3, [1,2,3])
→ add [1,2,3]
pop → [1,2]
pop → [1]
i=2: backtrack(3, [1,3])
→ add [1,3]
pop → [1]
pop → []
i=1: backtrack(2, [2])
→ add [2]
i=2: backtrack(3, [2,3])
→ add [2,3]
pop → [2]
pop → []
i=2: backtrack(3, [3])
→ add [3]
pop → []
```

Result: [[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]]

Curated Problems

#	Problem	Difficulty	Variant
1	Subsets (LC 78)	Medium	Generate all
2	Subsets II (LC 90)	Medium	Skip duplicates
3	Permutations (LC 46)	Medium	Generate all
4	Permutations II (LC 47)	Medium	Skip duplicates
5	Combination Sum (LC 39)	Medium	Reuse elements
6	Combination Sum II (LC 40)	Medium	No reuse + dedup
7	Palindrome Partitioning (LC 131)	Medium	Partition string
8	Word Search (LC 79)	Medium	Grid backtracking
9	N-Queens (LC 51)	Hard	Constraint satisfaction
10	Sudoku Solver (LC 37)	Hard	Constraint satisfaction

3.7 Dynamic Programming

Explanation

Solve complex problems by breaking them into overlapping subproblems. Store results to avoid recomputation (memoization top-down, or tabulation bottom-up).

When to Use

- **Optimal substructure** — optimal solution built from optimal sub-solutions

- **Overlapping subproblems** — same subproblems solved multiple times
- Keywords: "maximum", "minimum", "count ways", "is it possible"

Approach

1. Define state — what parameters describe a subproblem?
2. Define recurrence — how does state relate to smaller states?
3. Define base cases
4. Choose implementation — top-down (recursion + memo) or bottom-up (table)
5. Optimize space if possible

Variants

1D DP

```
# Climbing Stairs — dp[i] = ways to reach step i
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for i in range(3, n + 1):
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr
    return prev1

# House Robber — dp[i] = max money robbing houses [0..i]
def rob(nums):
    if not nums:
        return 0
    if len(nums) <= 2:
        return max(nums)
    prev2, prev1 = nums[0], max(nums[0], nums[1])
    for i in range(2, len(nums)):
        curr = max(prev1, prev2 + nums[i])
        prev2 = prev1
        prev1 = curr
    return prev1
```

2D DP

```

# Longest Common Subsequence
def lcs(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

# Unique Paths (grid)
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    return dp[m-1][n-1]

# Edit Distance
def min_distance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

```

Knapsack Variants

```

# 0/1 Knapsack
def knapsack_01(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i-1][w] # don't take item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1])
    return dp[n][capacity]

# Unbounded Knapsack (Coin Change)
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i and dp[i - coin] != float('inf'):
                dp[i] = min(dp[i], dp[i - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

# Coin Change 2 - Count Ways
def coin_change_ways(amount, coins):
    dp = [0] * (amount + 1)
    dp[0] = 1
    for coin in coins: # iterate coins in outer loop to avoid duplicates
        for i in range(coin, amount + 1):
            dp[i] += dp[i - coin]
    return dp[amount]

```

Longest Increasing Subsequence (LIS)

```

# O(n^2) DP
def lis_dp(nums):
    n = len(nums)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# O(n log n) with Binary Search
import bisect

def lis_binary_search(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)

```

State Machine DP (Stock Problems)

```

# Best Time to Buy and Sell Stock with Cooldown
def max_profit_cooldown(prices):
    n = len(prices)
    if n < 2:
        return 0
    # States: held, sold, cooldown
    held = -prices[0]
    sold = 0
    cooldown = 0
    for i in range(1, n):
        new_held = max(held, cooldown - prices[i])
        new_sold = held + prices[i]
        new_cooldown = max(cooldown, sold)
        held, sold, cooldown = new_held, new_sold, new_cooldown
    return max(sold, cooldown)

# Best Time to Buy and Sell Stock with K transactions
def max_profit_k(k, prices):
    n = len(prices)
    if not prices or k == 0:
        return 0
    if k >= n // 2: # unlimited transactions
        return sum(max(prices[i+1] - prices[i], 0) for i in range(n - 1))
    dp = [[0] * n for _ in range(k + 1)]
    for t in range(1, k + 1):
        max_diff = -prices[0]
        for d in range(1, n):
            dp[t][d] = max(dp[t][d-1], prices[d] + max_diff)
            max_diff = max(max_diff, dp[t-1][d] - prices[d])
    return dp[k][n-1]

```

Bitmask DP

```
# Traveling Salesman Problem (TSP) – visit all nodes with minimum cost
def tsp(dist):
    n = len(dist)
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0 # start at node 0

    for mask in range(1 << n):
        for u in range(n):
            if not (mask & (1 << u)):
                continue
            for v in range(n):
                if mask & (1 << v):
                    continue
                new_mask = mask | (1 << v)
                dp[new_mask][v] = min(dp[new_mask][v], dp[mask][u] + dist[u][v])

    full_mask = (1 << n) - 1
    return min(dp[full_mask][u] + dist[u][0] for u in range(n))
```

Interval DP

```
# Burst Balloons
def max_coins(nums):
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):
        for left in range(n - length):
            right = left + length
            for k in range(left + 1, right):
                dp[left][right] = max(
                    dp[left][right],
                    dp[left][k] + dp[k][right] + nums[left] * nums[k] * nums[right]
                )
    return dp[0][n - 1]
```

Curated Problems

#	Problem	Difficulty	Type
1	Climbing Stairs (LC 70)	Easy	1D
2	House Robber (LC 198)	Medium	1D
3	Coin Change (LC 322)	Medium	Unbounded Knapsack
4	Longest Increasing Subsequence (LC 300)	Medium	LIS
5	Longest Common Subsequence (LC 1143)	Medium	2D
6	Word Break (LC 139)	Medium	1D
7	Unique Paths (LC 62)	Medium	2D Grid
8	Edit Distance (LC 72)	Medium	2D
9	Target Sum (LC 494)	Medium	Knapsack
10	Partition Equal Subset Sum (LC 416)	Medium	0/1 Knapsack
11	Best Time — Cooldown (LC 309)	Medium	State Machine
12	Best Time — K Transactions (LC 188)	Hard	State Machine
13	Burst Balloons (LC 312)	Hard	Interval
14	Regular Expression Matching (LC 10)	Hard	2D
15	Palindrome Partitioning II (LC 132)	Hard	1D + 2D

3.8 Greedy

Explanation

Make the locally optimal choice at each step, hoping it leads to a globally optimal solution. Unlike DP, greedy doesn't explore all possibilities.

When to Use

- Problem has **greedy choice property** — local optimum leads to global optimum
- Interval scheduling, activity selection
- Huffman coding
- Problems where sorting + greedy scan works

Template Code

```

# Interval Scheduling – Maximum Non-overlapping Intervals
def erase_overlap_intervals(intervals):
    intervals.sort(key=lambda x: x[1]) # sort by end time
    count = 0
    end = float('-inf')
    for s, e in intervals:
        if s >= end:
            end = e
        else:
            count += 1
    return count

# Activity Selection – Maximum Activities
def max_activities(activities):
    activities.sort(key=lambda x: x[1])
    selected = [activities[0]]
    for i in range(1, len(activities)):
        if activities[i][0] >= selected[-1][1]:
            selected.append(activities[i])
    return selected

# Jump Game – Can reach end?
def can_jump(nums):
    max_reach = 0
    for i in range(len(nums)):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + nums[i])
    return True

# Jump Game II – Minimum jumps
def jump(nums):
    jumps = 0
    current_end = 0
    farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            jumps += 1
            current_end = farthest
    return jumps

# Gas Station
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost):
        return -1
    start = 0
    tank = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start = i + 1
            tank = 0
    return start

```

Curated Problems

#	Problem	Difficulty	Pattern
1	Jump Game (LC 55)	Medium	Greedy scan
2	Jump Game II (LC 45)	Medium	BFS-like greedy
3	Non-overlapping Intervals (LC 435)	Medium	Sort by end
4	Meeting Rooms II (LC 253)	Medium	Sort + heap
5	Gas Station (LC 134)	Medium	Circular scan
6	Task Scheduler (LC 621)	Medium	Greedy + math
7	Partition Labels (LC 763)	Medium	Last occurrence
8	Hand of Straights (LC 846)	Medium	Counter + sort
9	Merge Triplets (LC 1899)	Medium	Filter valid
10	Valid Parenthesis String (LC 678)	Medium	Track min/max open

3.9 Divide and Conquer

Explanation

Break problem into smaller subproblems, solve each recursively, then combine results.

When to Use

- Sorting (merge sort, quick sort)
- Selection (quick select for kth element)
- Binary search variants
- Problems on ranges

Template Code

```

# Merge Sort –  $O(n \log n)$ , stable
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Quick Select –  $O(n)$  average for kth smallest
import random

def quick_select(nums, k):
    """Find kth smallest element (0-indexed)."""
    if len(nums) == 1:
        return nums[0]
    pivot = random.choice(nums)
    left = [x for x in nums if x < pivot]
    mid = [x for x in nums if x == pivot]
    right = [x for x in nums if x > pivot]
    if k < len(left):
        return quick_select(left, k)
    elif k < len(left) + len(mid):
        return pivot
    else:
        return quick_select(right, k - len(left) - len(mid))

# Count Inversions using Merge Sort
def count_inversions(arr):
    if len(arr) <= 1:
        return arr, 0
    mid = len(arr) // 2
    left, left_inv = count_inversions(arr[:mid])
    right, right_inv = count_inversions(arr[mid:])
    merged = []
    inversions = left_inv + right_inv
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            inversions += len(left) - i
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged, inversions

```

Curated Problems

#	Problem	Difficulty	Pattern
1	Sort an Array (LC 912)	Medium	Merge/Quick Sort
2	Kth Largest Element (LC 215)	Medium	Quick Select
3	Merge k Sorted Lists (LC 23)	Hard	Merge
4	Count of Smaller Numbers After Self (LC 315)	Hard	Modified Merge Sort
5	Reverse Pairs (LC 493)	Hard	Modified Merge Sort

3.10 Topological Sort

Explanation

Linear ordering of vertices in a DAG such that for every edge (u, v) , u comes before v . Think of it as task scheduling with prerequisites.

When to Use

- Course prerequisites / dependency resolution
- Build systems (Makefile order)
- Detecting cycles in directed graphs

Template Code

```

from collections import deque, defaultdict

# Kahn's Algorithm (BFS-based)
def topological_sort_kahn(num_nodes, edges):
    graph = defaultdict(list)
    in_degree = [0] * num_nodes
    for u, v in edges:
        graph[u].append(v)
        in_degree[v] += 1

    queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(order) != num_nodes:
        return [] # cycle detected
    return order

# DFS-based Topological Sort
def topological_sort_dfs(num_nodes, edges):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)

    visited = set()
    rec_stack = set()
    result = []

    def dfs(node):
        visited.add(node)
        rec_stack.add(node)
        for neighbor in graph[node]:
            if neighbor in rec_stack:
                return False # cycle
            if neighbor not in visited:
                if not dfs(neighbor):
                    return False
        rec_stack.remove(node)
        result.append(node)
        return True

    for i in range(num_nodes):
        if i not in visited:
            if not dfs(i):
                return [] # cycle
    return result[::-1]

```

Dry Run

Kahn's Algorithm:
Nodes: 0,1,2,3,4,5
Edges: (5,2), (5,0), (4,0), (4,1), (2,3), (3,1)

Initial in_degree: [2, 2, 1, 1, 0, 0]
Queue: [4, 5]

Pop 4 → order=[4], reduce 0(→1), 1(→1)
Pop 5 → order=[4,5], reduce 2(→0), 0(→1)
Queue: [2]
Pop 2 → order=[4,5,2], reduce 3(→0)
Queue: [3]
Pop 3 → order=[4,5,2,3], reduce 1(→0)
Pop 0 → order=[4,5,2,3,0]
Pop 1 → order=[4,5,2,3,0,1]

Result: [4, 5, 2, 3, 0, 1]

Curated Problems

#	Problem	Difficulty	Pattern
1	Course Schedule (LC 207)	Medium	Cycle detection
2	Course Schedule II (LC 210)	Medium	Topo sort
3	Alien Dictionary (LC 269)	Hard	Build graph + sort
4	Minimum Height Trees (LC 310)	Medium	Leaf trimming
5	Parallel Courses (LC 1136)	Medium	Level-based topo sort

3.11 Shortest Path Algorithms

Dijkstra's Algorithm

For **weighted graphs with non-negative weights**. Uses a min-heap (priority queue).

```
import heapq
from collections import defaultdict

def dijkstra(graph, start, n):
    """Returns shortest distance from start to all nodes."""
    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)] # (distance, node)

    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue # skip outdated entry
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

Time: $O((V + E) \log V)$ with binary heap.

Bellman-Ford Algorithm

For graphs with **negative weights** (but no negative cycles). Can detect negative cycles.

```
def bellman_ford(edges, n, start):
    dist = [float('inf')] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # Check for negative cycle
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None # negative cycle
    return dist
```

Time: $O(V * E)$

Floyd-Warshall Algorithm

All-pairs shortest path. Works with negative weights (no negative cycles).

```
def floyd_warshall(n, edges):
    dist = [[float('inf')] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    return dist
```

Time: $O(V^3)$

Algorithm	Time	Neg Weights	Use Case
Dijkstra	$O((V+E)\log V)$	No	Single source, non-negative
Bellman-Ford	$O(VE)$	Yes	Single source, detect neg cycles
Floyd-Warshall	$O(V^3)$	Yes	All pairs

3.12 Bit Manipulation

Explanation

Operate directly on binary representations of integers. Extremely fast (single CPU instruction) and space-efficient.

Key Operations

```

# Basics
x & y    # AND: both bits 1
x | y    # OR: at least one bit 1
x ^ y    # XOR: exactly one bit 1
~x       # NOT: flip all bits
x << n   # Left shift: multiply by 2^n
x >> n   # Right shift: divide by 2^n

# Common tricks
x & 1    # Check if odd
x & (x - 1) # Remove lowest set bit (Brian Kernighan's)
x | (x + 1) # Set lowest unset bit
x & (-x)   # Isolate lowest set bit
x ^ x     # Always 0 (self-XOR)
x ^ 0     # Always x

# Count set bits
def count_bits(n):
    count = 0
    while n:
        n &= n - 1 # remove lowest set bit
        count += 1
    return count
# Or simply: bin(n).count('1')

# Check if power of 2
def is_power_of_2(n):
    return n > 0 and (n & (n - 1)) == 0

# XOR tricks
# a ^ a = 0, a ^ 0 = a
# Find single number (every other appears twice)
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num
    return result

# Generate all subsets using bitmask
def subsets_bitmask(nums):
    n = len(nums)
    result = []
    for mask in range(1 << n):
        subset = [nums[i] for i in range(n) if mask & (1 << i)]
        result.append(subset)
    return result

# Swap without temp
a, b = a ^ b, a ^ b # NOT Pythonic, but demonstrates XOR
a = a ^ b # actually simpler: a, b = b, a

```

Curated Problems

#	Problem	Difficulty	Pattern
1	Single Number (LC 136)	Easy	XOR
2	Number of 1 Bits (LC 191)	Easy	Brian Kernighan
3	Counting Bits (LC 338)	Easy	DP + bit
4	Reverse Bits (LC 190)	Easy	Bit shifting
5	Missing Number (LC 268)	Easy	XOR
6	Sum of Two Integers (LC 371)	Medium	Bit arithmetic
7	Single Number II (LC 137)	Medium	Bit counting
8	Subsets (LC 78)	Medium	Bitmask

3.13 Math & Number Theory

Template Code

```
import math

# Sieve of Eratosthenes – find all primes up to n
def sieve(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            for j in range(i*i, n + 1, i):
                is_prime[j] = False
    return [i for i in range(n + 1) if is_prime[i]]

# Check if prime
def is_prime(n):
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

# GCD and LCM
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)

# Modular Exponentiation – compute (base^exp) % mod
def mod_pow(base, exp, mod):
    result = 1
    base %= mod
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        exp //= 2
        base = (base * base) % mod
    return result

# Modular Inverse (when mod is prime)
def mod_inverse(a, mod):
    return mod_pow(a, mod - 2, mod) # Fermat's little theorem

# Pascal's Triangle for nCr
def generate_pascal(n):
    triangle = [[1]]
    for i in range(1, n):
        row = [1]
        for j in range(1, i):
            row.append(triangle[i-1][j-1] + triangle[i-1][j])
        row.append(1)
        triangle.append(row)
    return triangle
```

Curated Problems

#	Problem	Difficulty	Pattern
1	Count Primes (LC 204)	Medium	Sieve
2	Pow(x, n) (LC 50)	Medium	Fast exponentiation
3	Happy Number (LC 202)	Easy	Cycle detection
4	Plus One (LC 66)	Easy	Carry
5	Factorial Trailing Zeroes (LC 172)	Medium	Count factors of 5
6	Rotate Image (LC 48)	Medium	Matrix math
7	Spiral Matrix (LC 54)	Medium	Simulation

3.14 String Algorithms

KMP (Knuth-Morris-Pratt) — $O(n + m)$

Pattern matching without backtracking. Builds a failure function (LPS array).

```
def kmp_search(text, pattern):
    """Find all occurrences of pattern in text."""
    n, m = len(text), len(pattern)
    if m == 0:
        return []

    # Build LPS (Longest Proper Prefix which is also Suffix)
    lps = [0] * m
    length = 0
    i = 1
    while i < m:
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length != 0:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1

    # Search
    results = []
    i = j = 0
    while i < n:
        if text[i] == pattern[j]:
            i += 1
            j += 1
        if j == m:
            results.append(i - j)
            j = lps[j - 1]
        elif i < n and text[i] != pattern[j]:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return results
```

Rabin-Karp — $O(n + m)$ average

Rolling hash for substring matching. Good for multi-pattern search.

```
def rabin_karp(text, pattern):
    n, m = len(text), len(pattern)
    if m > n:
        return []
    base = 256
    mod = 10**9 + 7

    # Compute hash of pattern and first window
    p_hash = 0
    t_hash = 0
    h = pow(base, m - 1, mod)
    for i in range(m):
        p_hash = (base * p_hash + ord(pattern[i])) % mod
        t_hash = (base * t_hash + ord(text[i])) % mod

    results = []
    for i in range(n - m + 1):
        if p_hash == t_hash:
            if text[i:i+m] == pattern: # verify to avoid hash collision
                results.append(i)
        if i < n - m:
            t_hash = (base * (t_hash - ord(text[i]) * h) + ord(text[i + m])) % mod
            t_hash = (t_hash + mod) % mod
    return results
```

Z-Algorithm — $O(n + m)$

Computes Z-array where $Z[i]$ = length of longest substring starting from i that matches a prefix of the string.

```
def z_function(s):
    n = len(s)
    z = [0] * n
    z[0] = n
    l, r = 0, 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] > r:
            l, r = i, i + z[i]
    return z

def z_search(text, pattern):
    combined = pattern + '$' + text
    z = z_function(combined)
    m = len(pattern)
    return [i - m - 1 for i in range(m + 1, len(combined)) if z[i] == m]
```

Part 4: Curated Problem List (150+ Problems)

Arrays & Hashing (20 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	1	Two Sum	Easy	Hash Map	Complement lookup	Google, Amazon, Facebook
2	217	Contains Duplicate	Easy	Hash Set	Set membership	Amazon, Apple
3	242	Valid Anagram	Easy	Counter	Char frequency	Amazon, Google
4	49	Group Anagrams	Med	Hash + Sort	Sorted string as key	Amazon, Facebook
5	347	Top K Frequent Elements	Med	Hash + Heap	Counter + nlargest	Amazon, Facebook
6	238	Product of Array Except Self	Med	Prefix/Suffix	Left then right pass	Amazon, Apple, Facebook
7	36	Valid Sudoku	Med	Hash Set	Row/col/box sets	Uber, Apple
8	271	Encode and Decode Strings	Med	Design	Length-prefix encoding	Google, Meta
9	128	Longest Consecutive Sequence	Med	Hash Set	Check if start of sequence	Google, Amazon
10	560	Subarray Sum Equals K	Med	Prefix Sum + Hash	Map prefix_sum → count	Facebook, Google
11	53	Maximum Subarray	Med	Kadane's	Reset if sum < 0	Amazon, Microsoft
12	56	Merge Intervals	Med	Sorting	Sort by start, merge	Google, Facebook
13	57	Insert Interval	Med	Merge	Find overlap position	Google, LinkedIn
14	15	3Sum	Med	Sort + Two Ptr	Fix one, two-pointer rest	Facebook, Amazon
15	11	Container With Most Water	Med	Two Pointers	Move shorter wall	Amazon, Google
16	42	Trapping Rain Water	Hard	Two Ptr/Stack	Min(left_max, right_max) - h	Amazon, Google, Goldman
17	41	First Missing Positive	Hard	Cyclic Sort	Place num at index num-1	Amazon, Microsoft
18	73	Set Matrix Zeroes	Med	In-place	Use first row/col as markers	Amazon, Facebook
19	48	Rotate Image	Med	Matrix	Transpose + reverse rows	Amazon, Apple
20	169	Majority Element	Easy	Boyer-Moore	Cancel different elements	Amazon, Google

Two Pointers (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	167	Two Sum II (Sorted)	Med	Opposite	Move based on sum	Amazon
2	15	3Sum	Med	Fix + Opposite	Sort, skip duplicates	Facebook, Amazon
3	11	Container With Most Water	Med	Opposite	Move shorter side	Amazon, Google
4	42	Trapping Rain Water	Hard	Opposite	Track left/right max	Amazon, Google
5	26	Remove Duplicates (Sorted)	Easy	Same Dir	Slow/fast	Facebook
6	283	Move Zeroes	Easy	Same Dir	Non-zero pointer	Facebook
7	75	Sort Colors	Med	Three Ptr	Dutch National Flag	Microsoft
8	977	Squares of Sorted Array	Easy	Opposite	Compare absolutes	Facebook
9	125	Valid Palindrome	Easy	Opposite	Skip non-alphanum	Facebook, Microsoft
10	234	Palindrome Linked List	Easy	Fast/Slow	Find mid, reverse, compare	Amazon

Sliding Window (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	643	Maximum Average Subarray I	Easy	Fixed	Sum first k, slide	Facebook
2	3	Longest Substring No Repeat	Med	Variable + Hash	Track last index	Amazon, Google, Facebook
3	424	Longest Repeating Char Replace	Med	Variable	Window - max_freq $\leq$ k	Google
4	567	Permutation in String	Med	Fixed + Hash	Counter comparison	Microsoft
5	76	Minimum Window Substring	Hard	Variable + Hash	Expand right, shrink left	Facebook, Google, Amazon
6	239	Sliding Window Maximum	Hard	Monotonic Deque	Decreasing deque	Amazon, Google
7	209	Min Size Subarray Sum	Med	Variable	Shrink when sum $\geq$ target	Facebook
8	904	Fruit Into Baskets	Med	Variable + Hash	At most 2 distinct	Google
9	1004	Max Consecutive Ones III	Med	Variable	At most k flips	Google
10	438	Find All Anagrams	Med	Fixed + Hash	Sliding counter	Amazon, Facebook

Stack (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	20	Valid Parentheses	Easy	Stack	Match pairs	Amazon, Google, Facebook
2	155	Min Stack	Med	Aux Stack	Track min alongside	Amazon, Google
3	150	Eval Reverse Polish Notation	Med	Stack	Push nums, pop for ops	Amazon, LinkedIn
4	739	Daily Temperatures	Med	Monotonic	Decreasing stack	Facebook, Google
5	853	Car Fleet	Med	Stack	Sort by position descending	Google
6	84	Largest Rectangle in Histogram	Hard	Monotonic	Increasing stack	Amazon, Google
7	394	Decode String	Med	Stack	Push partial results	Google, Amazon
8	735	Asteroid Collision	Med	Stack	Simulate	Uber
9	496	Next Greater Element I	Easy	Monotonic	Map value → next greater	Amazon
10	224	Basic Calculator	Hard	Stack	Handle signs + parens	Amazon, Google

Binary Search (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	704	Binary Search	Easy	Standard	lo, hi, mid	Amazon
2	35	Search Insert Position	Easy	Left Bound	bisect_left	Google
3	74	Search a 2D Matrix	Med	1D Search	Flatten mentally	Amazon
4	33	Search in Rotated Array	Med	Modified	Determine sorted half	Facebook, Amazon
5	153	Find Min in Rotated Array	Med	Modified	Compare mid with hi	Amazon, Microsoft
6	34	First and Last Position	Med	Two Searches	bisect_left + bisect_right	Facebook
7	875	Koko Eating Bananas	Med	Answer Space	Binary search on speed	Google, Facebook
8	981	Time Based Key-Value Store	Med	Upper Bound	Search timestamps	Google, Amazon
9	4	Median of Two Sorted Arrays	Hard	Partition	Binary search on shorter	Amazon, Google
10	410	Split Array Largest Sum	Hard	Answer Space	Minimize maximum sum	Google

Linked List (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	206	Reverse Linked List	Easy	Iterative	prev, curr, next	Amazon, Google
2	21	Merge Two Sorted Lists	Easy	Merge	Dummy head	Amazon, Google
3	141	Linked List Cycle	Easy	Fast/Slow	Floyd's	Amazon
4	19	Remove Nth From End	Med	Two Ptr	Gap of n	Facebook
5	143	Reorder List	Med	Mid+Rev+Merge	Three-step	Amazon, Facebook
6	138	Copy List Random Pointer	Med	Hash Map	Old→new mapping	Amazon, Facebook
7	2	Add Two Numbers	Med	Simulation	Carry propagation	Amazon, Google
8	146	LRU Cache	Med	Hash + DLL	O(1) get/put	Amazon, Google, Facebook
9	23	Merge k Sorted Lists	Hard	Heap	Min-heap of heads	Amazon, Facebook
10	25	Reverse Nodes in k-Group	Hard	Reverse	Count k, reverse, connect	Amazon, Microsoft

Trees (15 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	104	Maximum Depth	Easy	DFS	1 + max(left, right)	Amazon, Google
2	226	Invert Binary Tree	Easy	DFS	Swap children	Google, Amazon
3	100	Same Tree	Easy	DFS	Compare recursively	Amazon
4	572	Subtree of Another Tree	Easy	DFS	Check same_tree at each node	Amazon, Facebook
5	102	Level Order Traversal	Med	BFS	Queue + level size	Amazon, Facebook
6	98	Validate BST	Med	DFS	Pass min/max bounds	Amazon, Facebook
7	230	Kth Smallest in BST	Med	Inorder	Inorder gives sorted order	Amazon
8	235	LCA of BST	Med	BST	Split point	Facebook, Amazon
9	236	LCA of Binary Tree	Med	DFS	Return when found	Facebook, Amazon, Google
10	199	Right Side View	Med	BFS	Last node per level	Facebook, Amazon
11	105	Construct from Pre/In	Med	Recursion	Root splits inorder	Google, Amazon
12	208	Implement Trie	Med	Trie	Dict children + is_end	Amazon, Google
13	124	Max Path Sum	Hard	DFS	Track max through each node	Facebook, Amazon
14	297	Serialize/Deserialize	Hard	Preorder	Null markers + queue	Amazon, Google
15	543	Diameter of Binary Tree	Easy	DFS	Left depth + right depth	Facebook, Google

Tries (5 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	208	Implement Trie	Med	Basic	Dict of children	Amazon, Google
2	211	Add and Search Words	Med	Trie + DFS	'.' matches any	Facebook
3	212	Word Search II	Hard	Trie + Grid	Build trie, backtrack grid	Amazon, Google
4	648	Replace Words	Med	Prefix	Find shortest root	Uber
5	1268	Search Suggestions System	Med	Trie + Sort	Top 3 suggestions	Amazon

Heap / Priority Queue (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	215	Kth Largest Element	Med	Heap/QSelect	Min-heap size k	Facebook, Amazon
2	347	Top K Frequent Elements	Med	Hash + Heap	Counter + heapq	Amazon, Facebook
3	23	Merge k Sorted Lists	Hard	Min-Heap	Push heads, expand	Amazon, Facebook
4	295	Find Median from Stream	Hard	Two Heaps	Max-heap + min-heap	Amazon, Google
5	621	Task Scheduler	Med	Greedy + Heap	Max freq first	Facebook, Amazon
6	973	K Closest Points	Med	Heap	Distance priority	Amazon, Facebook
7	767	Reorganize String	Med	Heap	Alternate most frequent	Google, Amazon
8	355	Design Twitter	Med	Heap + Hash	Merge k feeds	Amazon, Twitter
9	378	Kth Smallest Sorted Matrix	Med	Heap	Multi-way merge	Amazon, Google
10	703	Kth Largest in Stream	Easy	Min-Heap	Maintain size k	Amazon

Backtracking (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	78	Subsets	Med	Generate	Include/exclude each	Amazon, Facebook
2	90	Subsets II	Med	Dedup	Sort + skip duplicates	Amazon
3	46	Permutations	Med	Generate	Swap or remaining list	Amazon, Facebook
4	47	Permutations II	Med	Dedup	Sort + skip same adj	Amazon
5	39	Combination Sum	Med	Reuse	Same index allowed	Amazon, Facebook
6	40	Combination Sum II	Med	No Reuse	i+1 + skip duplicates	Amazon
7	131	Palindrome Partitioning	Med	Partition	Check palindrome + recurse	Amazon
8	79	Word Search	Med	Grid	Mark visited, backtrack	Amazon, Facebook
9	51	N-Queens	Hard	Constraint	Track cols, diags	Amazon, Google
10	17	Letter Combinations of Phone	Med	Generate	Map digit → chars	Amazon, Google

Graphs (15 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	200	Number of Islands	Med	DFS/BFS	Flood fill	Amazon, Google, Facebook
2	133	Clone Graph	Med	DFS + Hash	Old→new mapping	Facebook, Google
3	695	Max Area of Island	Med	DFS	Count cells	Amazon, Google
4	207	Course Schedule	Med	Topo Sort	Cycle detection	Amazon, Facebook
5	210	Course Schedule II	Med	Topo Sort	Return order	Amazon, Facebook
6	417	Pacific Atlantic Water Flow	Med	Multi BFS	BFS from each ocean	Google, Amazon
7	323	Number of Components	Med	Union-Find	Count components	Amazon, Google
8	261	Graph Valid Tree	Med	Union-Find	n-1 edges + connected	Google, Amazon
9	127	Word Ladder	Hard	BFS	Shortest transform	Amazon, Facebook
10	269	Alien Dictionary	Hard	Topo Sort	Build graph from words	Facebook, Google
11	743	Network Delay Time	Med	Dijkstra	Weighted shortest path	Amazon, Google
12	787	Cheapest Flights K Stops	Med	Bellman-Ford	Modified with k limit	Amazon, Google
13	684	Redundant Connection	Med	Union-Find	Edge creating cycle	Google
14	130	Surrounded Regions	Med	BFS/DFS	Border-connected O's	Google, Amazon
15	994	Rotting Oranges	Med	Multi BFS	Multi-source BFS	Amazon, Google

Dynamic Programming (20 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	70	Climbing Stairs	Easy	1D	$dp[i] = dp[i-1] + dp[i-2]$	Amazon, Google
2	198	House Robber	Med	1D	$\max(\text{skip}, \text{rob} + \text{prev})$	Amazon, Google
3	213	House Robber II	Med	Circular	Two passes: $[0..n-2]$, $[1..n-1]$	Google
4	322	Coin Change	Med	Unbounded	Min coins for each amount	Amazon, Google
5	518	Coin Change 2	Med	Unbounded	Count ways — coin outer loop	Amazon
6	300	Longest Increasing Subseq	Med	LIS	$O(n \log n)$ with bisect	Amazon, Google
7	1143	Longest Common Subseq	Med	2D	Match $\rightarrow$ diagonal + 1	Amazon, Google
8	139	Word Break	Med	1D	$dp[i] = \text{any } dp[j] + \text{word match}$	Facebook, Amazon
9	62	Unique Paths	Med	2D Grid	$dp[i][j] = \text{up} + \text{left}$	Google, Amazon
10	72	Edit Distance	Med	2D	Insert/delete/replace	Google, Amazon
11	416	Partition Equal Subset Sum	Med	0/1 Knapsack	Target = $\text{sum}/2$	Facebook, Amazon
12	494	Target Sum	Med	Knapsack	Convert to subset sum	Facebook, Google
13	5	Longest Palindromic Substr	Med	2D/Expand	Expand around centers	Amazon, Google
14	647	Palindromic Substrings	Med	Expand	Count all palindromes	Facebook
15	152	Maximum Product Subarray	Med	Track min/max	Min can become max	Amazon
16	91	Decode Ways	Med	1D	1-digit + 2-digit choices	Facebook, Amazon
17	309	Best Time — Cooldown	Med	State Machine	held/sold/cooldown states	Amazon
18	312	Burst Balloons	Hard	Interval	Last balloon to burst	Google
19	10	Regular Expression Matching	Hard	2D	'.' and '*' cases	Facebook, Google
20	115	Distinct Subsequences	Hard	2D	Count ways t appears in s	Google

Greedy (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	55	Jump Game	Med	Scan	Track max reachable	Amazon
2	45	Jump Game II	Med	BFS-like	Current/farthest end	Amazon, Google
3	435	Non-overlapping Intervals	Med	Sort	Sort by end time	Google
4	253	Meeting Rooms II	Med	Sort + Heap	Track ongoing meetings	Amazon, Google, Facebook
5	134	Gas Station	Med	Circular	If total gas $\geq$ total cost, solution exists	Amazon
6	763	Partition Labels	Med	Last Occur	Expand to include all chars	Amazon
7	621	Task Scheduler	Med	Math/Greedy	$(\text{max_freq}-1) * (n+1) + \text{count}$	Facebook
8	846	Hand of Straights	Med	Counter+Sort	Build consecutive groups	Google
9	1899	Merge Triplets	Med	Filter	Check which triplets can contribute	Google
10	678	Valid Parenthesis String	Med	Min/Max	Track range of open parens	Amazon, Google

Intervals (5 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	56	Merge Intervals	Med	Sort	Sort by start	Google, Facebook
2	57	Insert Interval	Med	Scan	Find overlap region	Google
3	435	Non-overlapping Intervals	Med	Sort End	Count overlaps	Google
4	252	Meeting Rooms	Easy	Sort	Check any overlap	Amazon
5	253	Meeting Rooms II	Med	Heap	Track end times	Amazon, Google

Math & Bit Manipulation (10 Problems)

#	LC #	Problem	Diff	Pattern	Hint	Frequency
1	136	Single Number	Easy	XOR	$a \oplus a = 0$, $a \oplus 0 = a$	Amazon
2	191	Number of 1 Bits	Easy	Kernighan	$n \& (n-1)$ clears lowest bit	Apple
3	338	Counting Bits	Easy	DP	$dp[i] = dp[i >> 1] + (i \& 1)$	Google
4	268	Missing Number	Easy	XOR/Math	XOR indices and values	Amazon
5	190	Reverse Bits	Easy	Shift	Build result bit by bit	Apple
6	371	Sum of Two Integers	Med	Bit Math	$carry = a \& b$, $sum = a \oplus b$	Facebook
7	7	Reverse Integer	Med	Math	Check overflow	Amazon
8	50	Pow(x, n)	Med	Fast Exp	Square and multiply	Facebook, Google
9	202	Happy Number	Easy	Cycle	Floyd's or set	Google
10	204	Count Primes	Med	Sieve	Sieve of Eratosthenes	Amazon

Part 5: Study Plan

5.1 Twelve-Week Structured Plan

Week 1: Foundations & Arrays

Day	Topic	Tasks
Mon	Python Review	Review Python internals, practice list/dict operations
Tue	Big O & Arrays	Study time/space complexity, solve LC 1, 217, 242
Wed	Arrays cont.	Solve LC 238, 53, 169
Thu	Arrays cont.	Solve LC 15, 11, 56
Fri	Arrays hard	Solve LC 42, 41
Sat	Review	Revisit all problems without looking at solutions
Sun	Rest/Light review	Read Python <code>collections</code> module documentation

Week 2: Hashing & Two Pointers

Day	Topic	Tasks
Mon	Hash Maps	Study hash table internals, solve LC 49, 347
Tue	Hash Maps cont.	Solve LC 560, 128, 36
Wed	Two Pointers (opposite)	Learn pattern, solve LC 167, 125, 977
Thu	Two Pointers (same dir)	Solve LC 26, 283, 75
Fri	Two Pointers (fast/slow)	Solve LC 141, 142, 234
Sat	Mixed practice	Solve 3 random problems from weeks 1-2
Sun	Rest	Review notes

Week 3: Sliding Window & Binary Search

Day	Topic	Tasks
Mon	Fixed Sliding Window	Learn pattern, solve LC 643, 567
Tue	Variable Sliding Window	Solve LC 3, 209, 424
Wed	Advanced Sliding Window	Solve LC 76, 239
Thu	Binary Search basics	Learn template, solve LC 704, 35, 34
Fri	Binary Search advanced	Solve LC 33, 153, 875
Sat	BS on answer space	Solve LC 410, 981
Sun	Rest/Review	Revisit sliding window problems

Week 4: Linked Lists & Stacks

Day	Topic	Tasks
Mon	Linked Lists basics	Learn patterns, solve LC 206, 21
Tue	LL fast/slow & merge	Solve LC 143, 19, 141
Wed	LL advanced	Solve LC 138, 2, 146
Thu	Stack basics	Learn patterns, solve LC 20, 155, 150
Fri	Monotonic Stack	Solve LC 739, 496, 853
Sat	Stack hard	Solve LC 84, 394
Sun	Rest	Review all linked list patterns

Week 5: Trees (Part 1)

Day	Topic	Tasks
Mon	Tree traversals	Implement all 4 traversals, solve LC 104, 226
Tue	BST properties	Solve LC 98, 230, 235
Wed	Tree DFS	Solve LC 100, 572, 236
Thu	Tree BFS	Solve LC 102, 199, 543
Fri	Tree construction	Solve LC 105, 297
Sat	Tree hard	Solve LC 124
Sun	Rest	Review tree patterns

Week 6: Tries & Heaps

Day	Topic	Tasks
Mon	Trie implementation	Build trie, solve LC 208
Tue	Trie problems	Solve LC 211, 212
Wed	Heap basics	Learn heapq, solve LC 215, 703
Thu	Heap patterns	Solve LC 347, 973, 295
Fri	Heap + Greedy	Solve LC 621, 767
Sat	Merge k pattern	Solve LC 23, 378
Sun	Rest	Review heap patterns

Week 7: Graphs (Part 1)

Day	Topic	Tasks
Mon	Graph representation	Build adjacency list, BFS, DFS
Tue	Grid BFS/DFS	Solve LC 200, 695, 130
Wed	Graph BFS	Solve LC 994, 127
Thu	Clone & Components	Solve LC 133, 323
Fri	Union-Find	Implement UF, solve LC 261, 684
Sat	Multi-source BFS	Solve LC 417
Sun	Rest	Review graph patterns

Week 8: Graphs (Part 2) & Topological Sort

Day	Topic	Tasks
Mon	Topological Sort	Learn Kahn's, solve LC 207, 210
Tue	Advanced topo	Solve LC 269
Wed	Shortest Path	Learn Dijkstra, solve LC 743
Thu	Weighted graphs	Solve LC 787
Fri	Graph review	Revisit all graph problems
Sat	Mixed graph practice	3 random graph problems
Sun	Rest	Review algorithms

Week 9: Dynamic Programming (Part 1)

Day	Topic	Tasks
Mon	DP basics, 1D	Learn approach, solve LC 70, 198, 213
Tue	1D DP cont.	Solve LC 139, 91, 152
Wed	2D DP	Solve LC 62, 1143, 72
Thu	Knapsack	Solve LC 322, 518, 416
Fri	Knapsack cont.	Solve LC 494
Sat	LIS	Solve LC 300 (both $O(n^2)$ and $O(n \log n)$)
Sun	Rest	Review DP framework

Week 10: Dynamic Programming (Part 2)

Day	Topic	Tasks
Mon	Palindrome DP	Solve LC 5, 647
Tue	State Machine DP	Solve LC 309, 188
Wed	Interval DP	Solve LC 312
Thu	Hard DP	Solve LC 10, 115
Fri	DP review	Revisit hardest problems
Sat	Mixed DP practice	3 random DP problems
Sun	Rest	Study DP patterns summary

Week 11: Backtracking & Greedy

Day	Topic	Tasks
Mon	Subsets & Perms	Solve LC 78, 46, 90, 47
Tue	Combinations	Solve LC 39, 40, 17
Wed	Constraint problems	Solve LC 79, 131, 51
Thu	Greedy basics	Solve LC 55, 45, 134
Fri	Greedy intervals	Solve LC 435, 253, 763
Sat	Greedy advanced	Solve LC 621, 678
Sun	Rest	Review patterns

Week 12: Advanced Topics & Mock Interviews

Day	Topic	Tasks
Mon	Bit Manipulation	Solve LC 136, 191, 338, 268
Tue	Math problems	Solve LC 50, 202, 204
Wed	String algorithms	Study KMP, solve string problems
Thu	Mock Interview 1	2 medium problems in 45 minutes
Fri	Mock Interview 2	1 medium + 1 hard in 60 minutes
Sat	Weak area review	Revisit topics you struggled with
Sun	Final review	Go through all patterns one more time

5.2 How to Approach a Problem You've Never Seen (5-Step Framework)

Step 1: Understand (2-3 minutes)

- Read the problem statement **twice**
- Identify inputs, outputs, and constraints
- Walk through examples manually
- Ask clarifying questions (in an interview):

- "Can the input be empty?"
- "Are there duplicates?"
- "Is the input sorted?"
- "What should I return if no solution exists?"

Step 2: Match (1-2 minutes)

- What data structure does this problem remind you of?
- What pattern fits? (see decision tree in Part 6)
- Look for keyword signals:
- "Sorted array" → Binary Search or Two Pointers
- "All permutations/subsets" → Backtracking
- "Minimum/maximum path/cost" → DP or Greedy or BFS
- "Connected components" → Union-Find or DFS
- "Top K" → Heap
- "Substring" → Sliding Window
- "Tree levels" → BFS

Step 3: Plan (3-5 minutes)

- Write pseudocode or explain approach in plain English
- State the time and space complexity before coding
- Get interviewer buy-in ("Does this approach sound good?")

Step 4: Code (15-20 minutes)

- Write clean, readable code
- Use meaningful variable names
- Add brief comments for non-obvious logic
- Test as you code with the given examples

Step 5: Verify (3-5 minutes)

- Trace through your code with the examples
- Think about edge cases:
- Empty input
- Single element
- All same values
- Very large input (does complexity hold?)
- Negative numbers
- Fix bugs calmly if found

5.3 What to Do When You're Stuck (Decision Tree)

```
Are you stuck?
├─ Can't understand the problem?
│  ├── Re-read the problem statement
│  ├── Draw the example step by step
│  └─ Ask "what is the simplest test case?"
├─ Can't think of an approach?
│  ├── What's the brute force? Start there.
│  ├── Can brute force be optimized? (Hash map, sort, binary search)
│  ├── What pattern does this remind you of? (See pattern decision tree)
│  ├── Can you solve a simpler version first?
│  └─ Work backwards from the answer – what info do you need?
├─ Know the approach but can't code it?
│  ├── Write pseudocode first
│  ├── Start with the base case
│  ├── Code the skeleton (function signature, loops, returns)
│  └─ Fill in the details
├─ Code doesn't work?
│  ├── Trace through with a small example step by step
│  ├── Print intermediate values (in practice)
│  ├── Check off-by-one errors (< vs <=, start vs start+1)
│  ├── Check base cases
│  └─ Check variable initialization
└─ In an interview?
   ├── Communicate: "I'm thinking about..."
   ├── Ask for a hint: "Can I get a small nudge?"
   ├── Time-box: Don't spend >5 min stuck silently
   └─ Show your thought process – partial credit matters
```

5.4 How to Practice Mock Interviews

Setup

1. Use platforms: **Pramp**, **Interviewing.io**, **LeetCode Mock**, or practice with a friend.
2. Time yourself strictly: 45 min for 2 problems (typical technical screen).
3. Speak out loud as you solve — interviewers evaluate your thought process.

Format

1. **Problem reading**: 2-3 minutes
2. **Approach discussion**: 5 minutes (explain before coding)
3. **Coding**: 15-20 minutes per problem
4. **Testing**: 3-5 minutes
5. **Optimization discussion**: 2-3 minutes

Tips

- Practice on a whiteboard or Google Doc (not IDE with autocomplete)
- Start with brute force, then optimize
- Always state time and space complexity
- If stuck, talk through your thinking — interviewers give hints to candidates who communicate
- After the mock: review the solution, understand what you missed

Weekly Schedule

- **2 mock interviews per week** (one with a partner, one solo timed)
 - **After each mock:** Write down what went well and what didn't
 - Track which patterns you struggle with
-

5.5 Behavioral Interview Prep (STAR Method)

The STAR Framework

- **Situation:** Set the context (where, when, what project)
- **Task:** What was your responsibility?
- **Action:** What did YOU specifically do? (Use "I", not "we")
- **Result:** What was the outcome? Quantify if possible.

Common Questions & Example Answers

"Tell me about a challenging technical problem you solved."

S: While working on an e-commerce platform, our search API response time degraded from 200ms to 3 seconds as the product catalog grew to 2 million items.

T: I was tasked with investigating and fixing the performance issue within one sprint.

A: I profiled the API and found that the database query was doing a full table scan. I implemented Elasticsearch for full-text search, added Redis caching for popular queries, and optimized the database schema with proper indexes.

R: Response time dropped to 50ms (98% improvement), and we could handle 10x more concurrent searches. The solution has been stable for 2 years.

"Tell me about a time you disagreed with a teammate."

S: During a code review, a senior engineer wanted to use a microservices architecture for a small internal tool.

T: I believed a monolithic approach would be more appropriate given the team size (3 people) and scope.

A: I prepared a comparison document showing the trade-offs: deployment complexity, debugging difficulty, and maintenance overhead. I presented it respectfully in a team meeting, acknowledging the benefits of microservices for larger systems.

R: The team agreed on a modular monolith approach — clean separation of concerns without the operational complexity. The project was delivered 2 weeks ahead of schedule.

"Tell me about a time you failed."

S: I pushed a database migration to production without testing it on a staging environment first.

T: The migration was supposed to add a new column, but it locked the users table for 15 minutes during peak hours.

A: I immediately rolled back the migration, communicated the issue to the team, and set up a proper staging environment. I also created a runbook for future migrations including off-peak scheduling and lock-free migration

techniques.

R: Zero data loss, but 15 minutes of degraded service. The runbook prevented similar incidents going forward, and the team adopted a mandatory staging-first policy.

Key Principles

- Prepare **6-8 stories** that cover: leadership, conflict, failure, technical challenge, teamwork, tight deadline
 - Each story can be adapted to answer multiple questions
 - Be specific — vague answers are red flags
 - Own your mistakes — interviewers value self-awareness
 - Quantify results whenever possible
-

5.6 System Design Basics (for Senior Roles)

Framework for System Design Interviews

1. **Clarify requirements** (5 min): Functional vs non-functional, scale estimates
2. **High-level design** (10 min): Core components, API design
3. **Detailed design** (15 min): Data models, algorithms, trade-offs
4. **Scale & bottlenecks** (10 min): Caching, sharding, load balancing

Key Concepts to Know

- **Load Balancing:** Round-robin, consistent hashing
- **Caching:** CDN, Redis/Memcached, cache invalidation strategies (TTL, write-through, write-behind)
- **Database:** SQL vs NoSQL, sharding (horizontal partitioning), replication (master-slave), CAP theorem
- **Message Queues:** Kafka, RabbitMQ — decoupling, async processing
- **Microservices:** Service discovery, API gateway, circuit breaker
- **Storage:** Blob storage (S3), file systems, CDN for static content

Common System Design Questions

1. Design URL Shortener (TinyURL)
2. Design Twitter/Instagram Feed
3. Design a Chat System (WhatsApp)
4. Design a Rate Limiter
5. Design a Web Crawler
6. Design YouTube/Netflix
7. Design Google Search Autocomplete
8. Design a Notification System

Recommended Resources

- "System Design Interview" by Alex Xu (Vol. 1 & 2)
- Grokking the System Design Interview (Educative)
- ByteByteGo (YouTube channel)

5.7 Resume Tips for Product-Based Companies

Format

- **One page** (even for experienced candidates)
- Clean, ATS-friendly format (no tables, no images)
- Consistent formatting (dates, bullet points)
- PDF format

Structure

1. **Contact Info:** Name, email, phone, LinkedIn, GitHub
2. **Education:** Degree, university, graduation date, GPA (if > 3.5)
3. **Experience:** Most recent first, 3-5 bullet points per role
4. **Projects:** 2-3 significant projects with tech stack
5. **Skills:** Languages, frameworks, tools (only what you can discuss in an interview)

Bullet Point Formula

[Action verb] + [What you did] + [Technology used] + [Quantifiable result]

Examples:

- "Designed and implemented a real-time data pipeline using Apache Kafka and Spark, processing 10M+ events daily with 99.9% uptime"
- "Reduced API latency by 60% by implementing Redis caching and optimizing PostgreSQL queries using proper indexing"
- "Built a CI/CD pipeline using GitHub Actions, reducing deployment time from 2 hours to 15 minutes"

Common Mistakes

- Too long (2+ pages)
 - Listing responsibilities instead of achievements
 - No quantification (use numbers!)
 - Including irrelevant skills
 - Typos or inconsistent formatting
 - Objective statement (outdated — use a summary only if senior)
-

Part 6: Quick Reference

6.1 Time/Space Complexity Cheat Sheet

Data Structure Operations

Data Structure	Access	Search	Insert	Delete	Space
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Dynamic Array	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^\dagger$	$O(1)^\dagger$	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Hash Table	—	$O(1)$	$O(1)$	$O(1)$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
BST (worst)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Min/Max Heap	$O(1)^\ddagger$	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Trie	—	$O(m)$	$O(m)$	$O(m)$	$O(N*m)$

* Amortized $\dagger$ At known position $\ddagger$ Peek only (min or max)

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes
Tim Sort (Python)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Graph Algorithms

Algorithm	Time	Space	Notes
BFS	$O(V + E)$	$O(V)$	Queue-based
DFS	$O(V + E)$	$O(V)$	Stack/recursion
Dijkstra	$O((V+E)\log V)$	$O(V)$	Min-heap, non-negative
Bellman-Ford	$O(VE)$	$O(V)$	Handles negative weights
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-pairs shortest path
Topological Sort	$O(V + E)$	$O(V)$	DAG only
Kruskal's MST	$O(E \log E)$	$O(V)$	Union-Find
Prim's MST	$O((V+E)\log V)$	$O(V)$	Min-heap
Union-Find ops	$O(\alpha(n))$	$O(n)$	Nearly constant

6.2 Python Syntax Cheat Sheet

```
# — CORE SYNTAX —
# Variables (dynamically typed)
x = 10
s = "hello"
lst = [1, 2, 3]
d = {"a": 1}
t = (1, 2)
st = {1, 2, 3}

# Multiple assignment
a, b, c = 1, 2, 3
a, b = b, a # swap

# Ternary
x = a if condition else b

# — STRINGS —
s.lower() s.upper() s.strip() s.split() s.replace(old, new)
s.startswith("he") s.endswith("lo") s.find("ll") # -1 if not found
s.isalpha() s.isdigit() s.isalnum()
ord('a') # 97 chr(97) # 'a'
''.join(list_of_strings)
f"x={x}, y={y:.2f}"

# — LISTS —
lst.append(x) lst.pop() lst.pop(0) lst.insert(i, x)
lst.extend([4,5]) lst.remove(val) lst.reverse()
lst.sort() lst.sort(key=lambda x: x[1], reverse=True)
sorted(lst) # returns new list
lst[::-1] # reversed copy
lst[start:stop:step]

# — DICTS —
d[key] = val d.get(key, default) d.setdefault(key, [])
d.keys() d.values() d.items()
d.pop(key) del d[key]
{k: v for k, v in iterable}

# — SETS —
s.add(x) s.remove(x) s.discard(x) # discard: no error if missing
s1 & s2 # intersection s1 | s2 # union
s1 - s2 # difference s1 ^ s2 # symmetric difference
x in s # O(1) lookup

# — BUILT-INS —
len() range() enumerate() zip() map() filter()
```

```

min() max() sum() abs() sorted() reversed()
any() all() isinstance() type()
int() float() str() list() tuple() set() dict()
bin(x) hex(x) oct(x) # int to binary/hex/octal string

# — LOOPS —————
for i in range(n):
for i, val in enumerate(lst):
for key, val in d.items():
for a, b in zip(lst1, lst2):
while condition:

# — FUNCTIONS —————
def func(a, b=10, *args, **kwargs):
    return result
lambda x: x**2

# — CLASSES —————
class MyClass:
    def __init__(self, val):
        self.val = val
    def method(self):
        return self.val

# — USEFUL PATTERNS —————
# Infinity
float('inf') float('-inf') math.inf

# Default dict
from collections import defaultdict
d = defaultdict(list) # or int, set

# Counter
from collections import Counter
c = Counter("aabbcc") # {'a':2, 'b':2, 'c':1}

# Heap
import heapq
heapq.heappush(h, val)
heapq.heappop(h)
heapq.heapify(lst)

# Deque
from collections import deque
dq = deque()
dq.append(x) dq.appendleft(x)
dq.pop() dq.popleft()

# Bisect
import bisect
bisect.bisect_left(arr, x)
bisect.insort(arr, x)

# Sort by custom key
intervals.sort(key=lambda x: (x[0], -x[1]))

# Recursion limit
import sys
sys.setrecursionlimit(10**6)

```

6.3 Common Patterns Decision Tree

Given the problem traits, which pattern to try:

Problem involves...

- └─ Sorted array / Finding a pair
 - └─ Find pair with target sum → TWO POINTERS (opposite)
 - └─ Find element → BINARY SEARCH
 - └─ Remove duplicates → TWO POINTERS (same direction)
- └─ Subarray / Substring
 - └─ Contiguous with constraint → SLIDING WINDOW
 - └─ Sum equals K → PREFIX SUM + HASH MAP
 - └─ Maximum sum → KADANE'S ALGORITHM
- └─ Tree
 - └─ Level-by-level → BFS
 - └─ Path/depth/sum → DFS (recursive)
 - └─ BST search/validate → BST property + DFS
 - └─ Serialize/construct → PREORDER + recursion
- └─ Graph
 - └─ Shortest path (unweighted) → BFS
 - └─ Shortest path (weighted) → DIJKSTRA
 - └─ Connected components → UNION-FIND or DFS
 - └─ Cycle detection → DFS (coloring) or UNION-FIND
 - └─ Task ordering → TOPOLOGICAL SORT
 - └─ Grid traversal → BFS/DFS with 4-directional movement
- └─ "Find all" / "Generate all"
 - └─ Subsets → BACKTRACKING (include/exclude)
 - └─ Permutations → BACKTRACKING (swap/remaining)
 - └─ Combinations → BACKTRACKING (choose k from n)
 - └─ Constraint satisfaction → BACKTRACKING (prune invalid)
- └─ Optimization (min/max/count ways)
 - └─ Overlapping subproblems? → DYNAMIC PROGRAMMING
 - └─ Greedy choice property? → GREEDY
 - └─ On a tree/DAG? → DFS/BFS + memo
 - └─ Small n (≤ 20)? → BITMASK DP or BACKTRACKING
- └─ Intervals
 - └─ Merge overlapping → SORT by start
 - └─ Max non-overlapping → SORT by end (GREEDY)
 - └─ Min rooms/resources → SORT + HEAP
- └─ Stack-like behavior
 - └─ Matching brackets → STACK
 - └─ Next greater/smaller → MONOTONIC STACK
 - └─ Expression evaluation → STACK
 - └─ Undo/backtrack → STACK
- └─ Top K / Kth element
 - └─ → HEAP (min-heap size k) or QUICK SELECT
- └─ Prefix matching / Word dictionary
 - └─ → TRIE
- └─ String matching / Pattern search
 - └─ → KMP / RABIN-KARP / Z-ALGORITHM
- └─ Bit-level operations / Single number
 - └─ → BIT MANIPULATION (XOR, masks)

Final Tips

1. **Consistency beats intensity** — 2 problems/day for 3 months beats 20/day for a week.
2. **Understand, don't memorize** — Know WHY the algorithm works, not just how.
3. **Revisit problems** — Solve the same problem again after 1 week. If you can't, you didn't learn it.

4. **Time yourself** — Real interviews have time pressure.
 5. **Write clean code** — Readability matters. Use good variable names.
 6. **Communicate** — In interviews, your thought process matters as much as the solution.
 7. **Learn from failures** — Every wrong answer teaches you something. Track your mistakes.
 8. **Stay calm** — It's okay to not know everything. Show how you think.
-

This guide was created for the Rightway-Intel team. Good luck with your interviews!